

Restrição do Tamanho de Kernel em CNNs para Aceleração Winograd

Rafael Silva de Souza
Curso de Engenharia de Computação
Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)
rafael.souza@inf.ufrgs.br

Mateus Grellert da Silva
Professor Adjunto, Orientador
Instituto de Informática

Universidade Federal do Rio Grande do Sul (UFRGS)
mateus.grellert@inf.ufrgs.br

Abstract—Aceleradores baseados no algoritmo de Winograd oferecem ganhos expressivos de eficiência para convoluções de kernel pequeno (2×2 , 3×3) [2], mas escalam mal para kernels grandes (5×5 , 11×11), comuns em arquiteturas clássicas como a AlexNet. Este trabalho investiga o custo de restringir o tamanho de kernel de uma CNN e se esse custo pode ser recuperado sem reintroduzir kernels grandes. Treinamos variantes da AlexNet no Tiny ImageNet-200 (64×64 , 200 classes) sob um pipeline FP32 $\rightarrow$ Quantization-Aware Training (QAT) $\rightarrow$ INT8, comparando um baseline de kernel irrestrito, uma restrição ingênua a 3×3 e uma família de mecanismos de compensação estrutural leve — blocos *bottleneck* ($1\times 1 \rightarrow 3\times 3 \rightarrow 1\times 1$), módulos *Fire* [4] e um atalho residual isolado — todos mantendo os kernels $\leq 3\times 3$. A variante com *bottleneck* (AlexNetBottleneck) atinge 44,62% de acurácia top-1 em FP32 com apenas 0,385M parâmetros (4,49 MB) — superando a restrição ingênua em mais de 4 pontos percentuais com $6\times$ menos parâmetros — e apresenta a menor queda de acurácia sob quantização INT8 do estudo ($-0,08$ p.p.). Uma ablação subsequente isola o efeito de um único atalho residual adicionado a um bloco *Fire*, sem nenhum parâmetro extra (AlexNetFireBypass): esse atalho captura aproximadamente 87% do ganho de acurácia FP32 obtido por uma arquitetura híbrida bem mais complexa que combina *stem* e *Fire+residual*, e supera essa arquitetura no regime INT8 (49,86% vs. 49,20%); nota-se que o AlexNetFireBypass usou orçamento de épocas superior. Medições diretas de hardware (GPU NVIDIA RTX 4060, rastreamento de kernel CUDA em nível de camada) indicam que a aceleração Winograd depende de convoluções densas ($\text{groups}=1$), não apenas do tamanho do kernel, e que compete com *tensor cores* em lotes maiores. Estendemos essas descobertas a duas frentes adicionais. Primeiro, testamos se a compensação estrutural e sua robustez à quantização transferem para predição densa — detecção SSD e segmentação DeepLabV3 em PASCAL VOC, com os três mesmos *backbones* — e encontramos transferência parcial e específica por mecanismo: AlexNetBottleneck lidera detecção (mAP 20,98%) e permanece estável sob INT8 em ambas as tarefas, mas AlexNetFire — quase indistinguível de AlexNetBottleneck em classificação — cai para menos da metade do mAP (8,94%), mostrando que paridade em classificação não garante paridade em predição densa. Segundo, avaliamos se atenção local (janelas ao estilo Swin, ViT/DeiT) reproduz o perfil de acurácia/eficiência/quantização das CNNs de kernel pequeno em sete modelos: o tamanho de janela reproduz monotonicamente a curva de restrição de kernel, nenhum dos sete aciona um kernel Winograd em nenhuma configuração — consistente com a natureza de multiplicação de matrizes densa da atenção — e, surpreendentemente, todos os sete *ganham* acurácia sob quantização INT8 (até +6,54 p.p.), superando os ganhos já observados nas CNNs de compensação; a destilação DeiT é o

modelo mais forte da família (46,38% FP32), mas às custas de $5\text{--}7\times$ mais parâmetros (2,76M) que as arquiteturas Pareto-ótimas deste estudo. Os resultados sugerem que compensação estrutural leve — e não kernels maiores — é uma via eficiente para recuperar acurácia em arquiteturas compatíveis com Winograd neste protocolo, e que mecanismos de custo zero (um único atalho residual) podem aproximar-se de arquiteturas híbridas mais elaboradas.

Index Terms—Winograd, convolução, quantização, redes neurais convolucionais, kernel bottleneck, módulo Fire, atalho residual, INT8, detecção de objetos, segmentação semântica, atenção local, transformers de visão

I. INTRODUÇÃO

O algoritmo de convolução de Winograd [2] reduz o número de multiplicações necessárias para convoluções de kernel pequeno, sendo a base de aceleradores de inferência amplamente usados em hardware embarcado. Sua vantagem, porém, desaparece rapidamente conforme o kernel cresce: o fator de redução de multiplicações frente à convolução direta satura numa constante em vez de crescer indefinidamente, e as matrizes de transformação tornam-se numericamente mal-condicionadas para kernels grandes [3], tornando kernels grandes (5×5 , 11×11) pouco atrativos nesses aceleradores.

Isso cria uma tensão direta com arquiteturas clássicas de visão computacional. A AlexNet [1], por exemplo, depende de kernels 11×11 e 5×5 em suas primeiras camadas para capturar contexto espacial amplo. Restringir ingenuamente esses kernels a 2×2 ou 3×3 — sem qualquer outra mudança arquitetural — reduz potencialmente a capacidade do modelo, pois cada camada enxerga uma vizinhança muito menor da imagem.

Este projeto investiga se essa perda de acurácia pode ser recuperada por meio de mecanismos de *compensação estrutural* que mantêm todos os kernels no regime Winograd-amigável ($\leq 3\times 3$), em vez de simplesmente devolver kernels grandes ao modelo. A hipótese central é que a restrição de tamanho de kernel, ainda que estruturalmente limitante, pode ser compensada por reorganizações arquiteturais de baixo custo — blocos *bottleneck*, módulos *Fire*, atalhos residuais — que redistribuem o poder representacional sem reintroduzir kernels grandes. Este relatório apresenta os resultados dessa investigação para dois mecanismos de compensação: blocos

bottleneck no estilo ResNet [5] (redução de canais $1 \times 1 \rightarrow$ convolução $3 \times 3 \rightarrow$ expansão 1×1) e módulos *Fire* no estilo SqueezeNet [4] (*squeeze* $1 \times 1 \rightarrow$ *expand* $1 \times 1/3 \times 3$), aplicados a uma AlexNet com kernels restritos a 3×3 . Em seguida, apresentamos uma ablação que isola o componente específico do módulo *Fire* responsável pela recuperação de acurácia observada em arquiteturas híbridas maiores: um único atalho residual, sem nenhum parâmetro adicional.

II. METODOLOGIA

A. Dados e configuração experimental

Todos os modelos foram avaliados no Tiny ImageNet-200 [9] (imagens 64×64 RGB, 200 classes, *split* 90/10 determinístico) sob o mesmo pipeline de três estágios: (i) treino FP32 — do zero para a grande maioria das variantes, exceto a baseline AlexNet irrestrita e os baselines externos ResNet18 [5]/MobileNetV2 [6] (Seção III-A), que partem de pesos pré-treinados em ImageNet-1K e são apenas ajustados (*fine-tuned*) para as 200 classes do Tiny ImageNet; (ii) *Quantization-Aware Training* (QAT), inicializado a partir do melhor checkpoint FP32, com fusão Conv-BN-ReLU e *fake quantization* nos módulos suportados [7] — as arquiteturas com atalho residual usam `nn.quantized.FloatFunctional` no lugar do operador `+` para o *add*, requisito do PyTorch para que a soma também seja quantizada corretamente; (iii) conversão para INT8 (*backend* `fbgemm`, inferência em CPU). Reportamos acurácia top-1 tanto para o modelo FP32 quanto para o modelo INT8 final, e a diferença entre ambos (*QAT drop*) como medida de robustez à quantização.

B. Arquiteturas comparadas

Sete variantes permitem isolar o efeito da restrição de kernel, o efeito do *head* GAP, o efeito de diferentes mecanismos de compensação, e o efeito de cada componente dentro de uma arquitetura híbrida. Todas são treinadas do zero sob o mesmo protocolo (Seção II-A), com exceção da baseline irrestrita, que é pré-treinada (ver abaixo):

- **AlexNet (irrestrita)** — arquitetura AlexNet padrão, kernels de até 11×11 , *pré-treinada em ImageNet-1K* e ajustada (*fine-tuned*) para 200 classes, usada como referência interna do experimento. Diferentemente de todas as demais variantes abaixo — treinadas do zero — essa baseline parte de pesos pré-treinados; comparações diretas de acurácia com ela confundem o efeito da restrição de kernel com o efeito do pré-treino (ver Seção IV).
- **AlexNet3x3-FC** — mesma arquitetura irrestrita, mas com todos os kernels convolucionais restritos a 3×3 e treinada do zero, mantendo a *head* totalmente conectada original. Serve de controle para isolar o efeito da restrição de kernel isoladamente do *head* GAP (usado apenas no próximo item) e do pré-treino.
- **AlexNet3x3-GAP** — mesma arquitetura, todos os kernels convolucionais restritos a 3×3 , com *head* de *global average pooling* (GAP) [16] no lugar de camadas totalmente conectadas.

- **AlexNetBottleneck** — kernels igualmente restritos a 3×3 , mas cada estágio é substituído por um bloco *bottleneck* ($1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$, razão de redução $4 \times$) seguido de GAP.
- **AlexNetFire** — kernels restritos a 3×3 , cada estágio substituído por um módulo *Fire* [4] (*squeeze* $1 \times 1 \rightarrow$ *expand* $1 \times 1 + 3 \times 3$, concatenados) seguido de GAP.
- **AlexNetFinalFireResidual** — arquitetura híbrida final que combina o módulo *Fire* acima com um novo *stem* 3×3 *stride-2* e atalhos residuais envolvendo cada estágio *Fire*.
- **AlexNetFireBypass** — arquitetura de ablação que parte do AlexNetFire original (mesmo *stem*, mesmos módulos *Fire*) e adiciona apenas um atalho residual por identidade em cada estágio, sem nenhum parâmetro extra em relação ao AlexNetFire base. Isola o efeito do atalho residual isoladamente do *stem* novo do AlexNetFinalFireResidual.

O bloco *bottleneck* e o módulo *Fire* preservam o campo receptivo efetivo da convolução 3×3 central, mas usam convoluções 1×1 para comprimir e depois expandir (ou concatenar) canais, aumentando a capacidade representacional por parâmetro sem introduzir nenhum kernel maior que 3×3 — mantendo a arquitetura inteiramente compatível com aceleração Winograd. O atalho residual por identidade usado no AlexNetFireBypass não introduz nenhuma convolução adicional, preservando trivialmente essa compatibilidade com Winograd.

Os Eixos 6 e 7 (Seções III-F e III-G) estendem esta comparação a dois contextos adicionais, descritos aqui pela mesma razão que as sete variantes acima: são arquiteturas comparadas neste estudo, não apenas resultados.

Predição densa (Eixo 6). Reaproveita, sem alterações no *backbone*, três das variantes acima — AlexNetBottleneck, AlexNetFire e AlexNetTV (a baseline de kernel irrestrito) — acopladas a uma *head* SSD [17] (detecção) ou DeepLabV3 [18] (segmentação); apenas a *head* muda entre classificação e predição densa, o *backbone* é idêntico ao já descrito.

Atenção local (Eixo 7). Sete arquiteturas testam se atenção local em janelas reproduz, em atenção, o perfil de restrição de kernel deste estudo:

- **swin_pico_{w2,w4,w8}** — Swin Transformer [22] em miniatura, dimensionada para entrada 64×64 (`patch_size=4`), com tamanho de janela de atenção local $\in \{2, 4, 8\}$ — o análogo, em atenção, do tamanho de kernel convolucional.
- **swin_pico_poolmixer** — mesma arquitetura acima, mas com a atenção substituída por um *pooling mixer* simples (sem mecanismo de atenção real); controle para isolar se o ganho observado vem da atenção em si ou apenas da arquitetura ao redor dela.
- **hybrid_bottleneck_swin** — *stem* convolucional que reaproveita o bloco *bottleneck* desta seção para o rebaixamento espacial inicial, seguido de estágios de atenção

em janela (Swin) para a mistura de contexto de longo alcance.

- **vit_tiny** — *Vision Transformer* (ViT) [21] compacto: a imagem é dividida em *patches* de tamanho fixo, cada *patch* é projetado linearmente em um *token*, e camadas de atenção multi-cabeça *global* padrão (não em janela — cada *token* atende a todos os demais, ao contrário do Swin acima) misturam informação entre todos os *tokens* simultaneamente, sem nenhuma convolução ou viés indutivo espacial. Dimensionado para 64×64 e treinado do zero com entropia cruzada padrão.
- **deit_tiny** — mesma arquitetura de **vit_tiny**, mas treinada com o esquema de destilação do DeiT (*Data-efficient image Transformer*) [23]: como transformers de visão carecem do viés indutivo convolucional e tipicamente exigem grandes volumes de dados ou pré-treino em larga escala para igualar CNNs, o DeiT treina o transformer para imitar diretamente os rótulos previstos (não apenas os rótulos verdadeiros) de um professor CNN já treinado — aqui, o MobileNetV2 pré-treinado e congelado (*baseline* externo do Eixo 1, Figura 1) — reduzindo a dependência de grandes quantidades de dados de treino.

Por não haver caminho INT8 estável para Layer-Norm/softmax/atenção neste *pipeline* de QAT (Seção II-A), essas camadas permanecem em FP32 durante a conversão nos sete modelos acima — apenas as camadas Linear/Conv convertem para INT8.

C. Perfilamento de hardware

As medições de latência (Eixo 3) foram feitas em uma única GPU NVIDIA RTX 4060 Laptop, em uma única sessão de coleta, cronometrando camadas Conv2d isoladas com `torch.cuda.synchronize` antes e depois de 200 iterações cronometradas (50 iterações de *warmup* descartadas). O algoritmo de convolução (Winograd, direto, ou GEMM via *tensor cores* TF32) é escolhido internamente pelo cuDNN a cada configuração — não é forçado pelo experimento — e identificado *a posteriori* por rastreamento do nome do *kernel* CUDA executado (`torch.profiler`). Por ser uma única sessão em uma laptop GPU com limite de potência, o *clock* sustentado observado varia entre sessões (deriva de até $2 \times$ em medições de controle no mesmo dia), então diferenças pequenas de latência devem ser lidas com essa incerteza em mente; ver limitações na Seção IV.

III. RESULTADOS

A. Eixo 1: Custo de Restrição de Kernel e Mecanismos de Compensação

Os três *baselines* externos da Figura 1 situam essa restrição num contexto mais amplo, fora do escopo Winograd-amigável ($\leq 3 \times 3$) deste estudo: MobileNetV2 pré-treinada atinge 58,0% de acurácia FP32 (28,75 MB, a maior do gráfico), ResNet18 pré-treinada 53,9% (129,21 MB), e VGG-Style — treinada do zero, sem pré-treino — 51,8% (27,58 MB). Do lado da restrição ingênua, o par completo citado

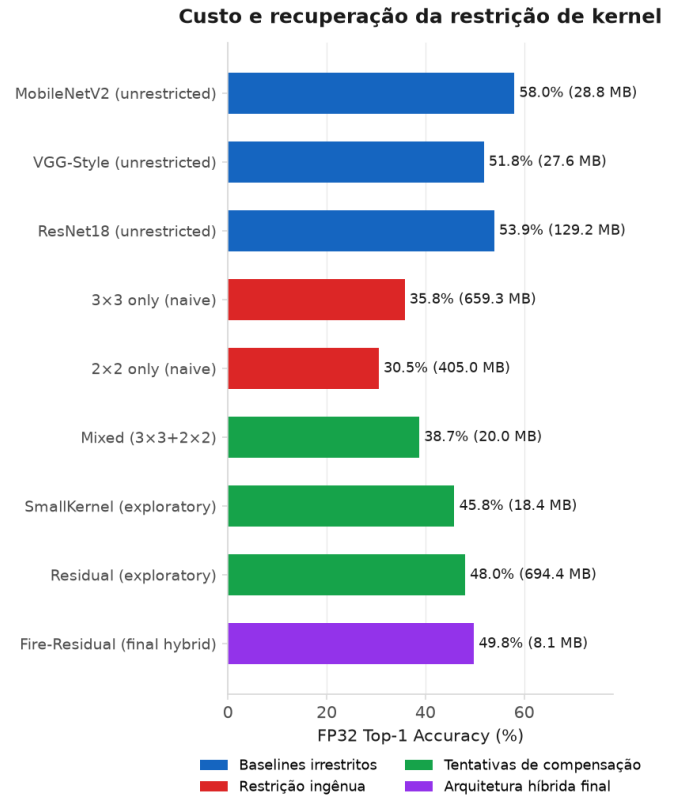


Figura 1. Acurácia FP32 (top-1) por arquitetura, contrastando *baselines* externos de kernel irrestrito (MobileNetV2, ResNet18, VGG-Style), restrição ingênua (AlexNet 3×3 e 2×2), tentativas exploratórias de compensação (Mixed, Residual puro — não detalhadas individualmente neste relatório; SmallKernel é retomado com números no Eixo 5) e a arquitetura híbrida final (Fire-Residual).

na legenda é AlexNet 3×3 -FC (35,79%, discutida abaixo) e AlexNet 2×2 -FC (30,53%, treinada do zero) — kernel 2×2 sozinho, sem compensação estrutural, fica abaixo até da baseline AlexNet pré-treinada de kernel irrestrito.

A Tabela I resume o custo de restringir kernels a 3×3 e a efetividade de dois mecanismos de compensação: blocos *bottleneck* e módulos *Fire*. Params e MACs medem eficiência de armazenamento e de computação separadamente — como o Eixo 3 mostra, eles nem sempre concordam sobre qual arquitetura é “mais eficiente”. AlexNet 3×3 -FC isola o efeito da restrição de kernel isoladamente do *head* GAP: mantém a *head* totalmente conectada da baseline, mas restringe kernels a 3×3 e treina do zero (diferente da baseline irrestrita, pré-treinada). Note que, apesar do kernel menor, AlexNet 3×3 -FC tem mais MACs que a baseline (222,3M vs. 94,9M); o *stem* 3×3 usa *stride* 2 em vez do *stride* 4 do *stem* 11×11 original, então os mapas de ativação permanecem maiores por mais camadas — um kernel menor por aplicação, mas aplicado sobre uma área espacial bem maior no total.

Restringir kernels a 3×3 custa acurácia? Não neste protocolo — o oposto da hipótese inicial. Isolando a restrição de kernel da troca de *head* e do pré-treino (AlexNet 3×3 -FC vs. baseline irrestrita, ambas com *head* totalmente conec-

Tabela I
CUSTO DE RESTRIÇÃO DE KERNEL (3×3) E MECANISMOS DE COMPENSAÇÃO

Modelo	Params (M)	MACs (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	ΔQAT (p.p.)
AlexNet (irrestrita, pré-treinada)	57,82	94,9	32,88	31,90	−0,98
AlexNet3x3-FC (do zero)	57,61	222,3	35,79	36,19	+0,40
AlexNet3x3-GAP	2,30	167,0	40,32	37,02	−3,29
AlexNetBottleneck	0,39	39,5	44,62	44,54	−0,08
AlexNetFire	0,52	177,7	43,98	44,30	+0,33

tada), a restrição *ganha* 2,9 p.p. (32,88%→35,79%). Uma explicação plausível é que a *head* totalmente conectada de 57M parâmetros da baseline overfita mais facilmente num dataset de imagens 64×64. Como só a baseline é pré-treinada, porém, o resultado não separa “custo da restrição de kernel” de “efeito do pré-treino”: é sugestivo, não conclusivo.

Qual mudança tem o efeito isolado mais claro? A troca de *head* para GAP (AlexNet3x3-GAP): mais 4,5 p.p. (35,79%→40,32%) com 25× menos parâmetros (57M → poucos milhares), consistente com redução de *overfitting* dada a dimensão reduzida do dataset.

Quanto a compensação estrutural recupera? O bloco *bottleneck* soma mais 4,3 p.p. sobre a variante GAP com 6× menos parâmetros, e praticamente elimina a perda sob INT8. O módulo *Fire* é um mecanismo alternativo: 0,52M parâmetros, 43,98% em FP32 e um *ganho* sob quantização (INT8 44,30%, +0,33 p.p.) — propriedade rara neste estudo, compatível com a hipótese de que a compressão *squeeze-expand* facilita a calibração de quantização, embora o mecanismo exato permaneça especulativo. Em MACs, porém, essa economia de parâmetros não se traduz em menos computação: o *Fire* usa 177,7M MACs, mais que a própria AlexNet3x3-GAP (167,0M) e 4,5× mais que o *Bottleneck* (39,5M) — apenas o *Bottleneck* reduz armazenamento e computação.

B. Eixo 2: Sinergia de Arquiteturas Híbridas

Investigamos combinações de mecanismos de compensação para maximizar a sinergia entre mecanismos estruturais. A Tabela II compara cinco combinações de *bottleneck*, *Fire*, atalhos residuais e convoluções *depthwise*.

A combinação *Fire* + Residual emerge como mais efetiva, ganhando 5,81 p.p. sobre o *Fire* de base (43,98% → 49,79%) — um efeito não linear que sugere sinergia entre compressão canais (*squeeze-expand*) e conexões residuais. Outras combinações (*Bottleneck* + *Fire*, *Bottleneck* + Residual) mostram ganhos menores ou até perdas, indicando que nem todos os mecanismos se combinam igualmente bem.

C. Eixo 3: Validação em Hardware da Aceleração Winograd

Investigamos se as arquiteturas do projeto realmente acionam aceleração Winograd em GPU, e sob quais condições. Duas fontes de evidência, de força bem diferente, são usadas: (i) rastreamento direto do nome do *kernel* CUDA executado pelo cuDNN — a evidência mais forte, pois identifica o algoritmo em vez de inferi-lo; e (ii) o *scaling* de latência com o tamanho do kernel, testado estatisticamente — uma evidência

indireta e, como mostrado abaixo, inconclusiva no regime que mais importa.

O rastreamento de *kernel* indica que o cuDNN seleciona um algoritmo identificado como Winograd especificamente para convoluções densas (*groups*=1) com kernel 3×3, nos *batches* 1 e 8 — mas não para convoluções *depthwise*, em nenhum tamanho de kernel (2 a 11) ou *batch* testado nesta GPU e backend. Esse é o resultado mais direto desta seção.

O teste de *scaling* teórico é mais fraco. A complexidade de Winograd não é assintoticamente menor que a da convolução direta: para um *tile* de saída fixo de $m \times m$ elementos, o número de multiplicações de Winograd, $(m + k - 1)^2$, e o da convolução direta, $m^2 k^2$, crescem ambos como $\Theta(k^2)$ com o tamanho do kernel k [2] — por exemplo, F(2×2,3×3) usa 16 multiplicações contra 36 da convolução direta (2,25×), e F(4×4,3×3) usa 36 contra 144 (4×) [2]; essa vantagem converge para um fator constante ($m^2 \times$) quando $k \rightarrow \infty$, em vez de crescer indefinidamente. Na prática, Winograd raramente é usado além de $k = 3$ (ocasionalmente $k = 5$) não por essa escala assintótica, mas porque as matrizes de transformação ficam numericamente mal-condicionadas para k grande [3]. A Figura 2 mostra a latência por FLOP medida para $k \in \{2, 3, 5, 7, 9, 11\}$. Dois regimes importam aqui: o *compute-bound* (*batch* = 64, GPU saturada, tempo dominado por cálculo real — exatamente onde uma redução de multiplicações deveria aparecer, e por isso a evidência primária) e o *overhead-bound* (*batch* = 1/8, GPU ociosa, tempo dominado por setup de kernel e sincronização).

Um teste de Wilcoxon pareado¹ [10] sobre a latência por FLOP ($k=3$ vs. $k=5$, denso) no regime *compute-bound* é **estatisticamente significativo, mas na direção oposta à esperada**: $p = 0,023$, $n = 8$, e a mediana do custo por FLOP em $k=3$ não é menor que em $k=5$. O teste rejeita a hipótese nula de nenhuma diferença sistemática, mas não corrobora a hipótese de que Winograd é mais eficiente por FLOP nesse regime. No *overhead-bound* há uma tendência favorável (mediana ~7% menor em $k=3$), sem significância estatística ($p = 0,82$, $n = 16$) — ali o overhead mascara qualquer ganho de redução de FLOP. Consistente com isso, o rastreamento de *kernel* mostra que em *batch* = 64 o cuDNN troca Winograd por um *kernel* TF32 de *tensor core* mesmo em $k=3$: no regime onde a redução de FLOP teria mais efeito, nem sempre há Winograd para medir. A leitura mais defensável não é que Winograd não ajuda, mas que *tensor*

¹Não-paramétrico, adequado para comparar duas condições medidas nas mesmas configurações sem assumir normalidade dos dados.

Tabela II
ABLAÇÃO DE ARQUITETURAS HÍBRIDAS: SINERGIA ENTRE MECANISMOS DE COMPENSAÇÃO

Arquitetura	Params (M)	MACs (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Δ QAT (p.p.)
AlexNetFire (base)	0,52	177,7	43,98	44,30	+0,33
Fire + Residual (AlexNetFinalFireResidual)	0,70	55,2	49,79	49,20	-0,59
Bottleneck + Fire (AlexNetFinalBottleneckFire)	0,51	47,3	42,29	44,00	+1,71
Bottleneck + Residual (AlexNetFinalBottleneckResidual)	0,57	54,2	45,10	45,98	+0,88
Depthwise + Fire (AlexNetFinalDepthwiseFire)	0,47	30,1	43,46	42,79	-0,67

cores competem com a vantagem de Winograd justamente no regime *compute-bound*, onde uma redução de multiplicações teria mais efeito — achado interessante para o acelerador FPGA que motiva este projeto (sem *tensor cores*), mas que este experimento em GPU não pode confirmar diretamente.

Para convoluções *depthwise*, o quadro é mais consistente: nenhuma evidência de Winograd foi encontrada em nenhum regime. No nível de camada, o mesmo teste de Wilcoxon [10] (*compute-bound*, *denso* vs. *groups=in_ch*) rejeita sinal Winograd para *depthwise* ($p = 0,0078$, $n = 8$). No nível de modelo completo, *alexnet_depthwiseseq* e *mobilenetv2* (85,0 e 24,9 GFLOP/s, mediana 54,9) ficam bem abaixo de quatro modelos densos comparáveis (*alexnet_bottleneck*, *alexnet_final_bottleneck_residual*, *alexnet_final_fire_residual*, *vgg_style*; mediana 112,5 GFLOP/s) — comparação de apenas $n = 2$ modelos *depthwise*, exploratória. O mais correto a afirmar é que, *nesta GPU e neste backend (cuDNN)*, convoluções *depthwise* não acionam um *kernel* Winograd, não que o algoritmo seja matematicamente incompatível com elas — a literatura descreve variantes de Winograd por canal para convoluções *depthwise* [13]; o que este experimento demonstra é que o cuDNN, nesta configuração, não as implementa.

Mapeamos também a fronteira de Pareto entre acurácia e latência para o conjunto completo de arquiteturas (Figura 3).

Quatro de cinco arquiteturas com convolução 3×3 densa (*AlexNetBottleneck*, *AlexNetFire*, *AlexNetFinalBottleneckResidual*, *AlexNetFinalFireResidual*) superam o baseline (*AlexNetTV*, irrestrito) na razão acurácia/latência. A latência em FP32 também é um preditor razoável do *ranking* de latência em INT8, mas só para convoluções densas (correlação de Spearman $\rho = 0,99$, $n = 24$); para *depthwise* a correlação é sensivelmente mais fraca ($\rho = 0,72$, $n = 24$) — o *backend* INT8 (fbgemm) tem *kernels* otimizados só para $3 \times 3/5 \times 5$ em convoluções agrupadas [8] e cai para um caminho genérico muito mais lento acima disso, o que quebra a correspondência FP32→INT8 nesse subconjunto.

Por fim, avaliamos a competição com *tensor cores* entre tamanhos de *batch*, no mesmo experimento acima: em *batch*=1 e 8 o cuDNN prefere Winograd em $k=3$; em *batch*=64 prefere um *kernel* TF32 de *tensor core* mesmo em $k=3$. Isso é uma limitação da GPU como plataforma de validação,

não uma medição do princípio em si: o acelerador FPGA que motiva este projeto não possui *tensor cores*, então essa competição específica não deveria ocorrer nele — mas essa é uma expectativa baseada na ausência de *tensor cores* no design do FPGA, não algo medido neste trabalho.

D. Eixo 4: Bypass Residual de Custo Zero

Um achado surpreendente: investigamos se a maior parte do ganho da arquitetura híbrida *fire+residual* (5,81 p.p. ganho sobre *Fire* de base) provém especificamente do atalho residual. Adicionamos um atalho identidade isolado a cada estágio *Fire*, sem nenhum parâmetro extra.

A Tabela III compara o *Fire* de base com essa ablação por atalho isolado (*FireBypass*) e a arquitetura híbrida completa.

O *AlexNetFireBypass* adiciona um único atalho residual por identidade a cada estágio *Fire*, sem nenhum parâmetro extra (0,52M em ambos os casos). Apesar dessa simplicidade, fecha 87% do ganho FP32 da arquitetura híbrida completa (5,05 de 5,81 p.p.) e a *supera* no regime INT8 (49,86% vs. 49,20%), mantendo ganho sob quantização (+0,84 p.p. vs. -0,59 p.p. do híbrido completo). Esse resultado isolado sugere que o atalho residual — não o novo *stem* da arquitetura híbrida — é o componente responsável pela maior parte do ganho de acurácia, e que pode ser adicionado a *custo estrutural zero*. Um adendo importante: o *AlexNetFireBypass* foi treinado com um orçamento de épocas maior que o restante do estudo (152 épocas efetivas, contra 77 do *AlexNetFinalFireResidual* — Tabela XI no apêndice), por ter sido executado no cluster PCAD da UFRGS sob um orçamento de época maior que o disponível localmente; parte do ganho de acurácia relatado aqui pode portanto também refletir esse treino mais longo, não apenas o atalho residual isoladamente. A Figura 4 situa esse resultado no conjunto completo de modelos do estudo.

A Figura 4 ordena todos os modelos de classificação do estudo segundo acurácia versus tamanho de *checkpoint*, complementando a visão latência-cêntrica da Figura 3. Quatro leituras se destacam.

Os modelos de atenção ocupam a base da fronteira FP32, mas por um motivo trivial. A fronteira é uma escada de 8 modelos, de *swin_pico_poolmixer* (2,66 MB, 28,21%) a *MobileNetV2* (28,75 MB, 58,00%), e seus dois primeiros degraus são modelos do Eixo 7 — apenas porque são os dois menores *checkpoints* de todo o estudo, não porque ofereçam um bom compromisso: o degrau seguinte,

RTX 4060 Laptop (local): layer latency per FLOP vs. kernel size (dense/groups=1, res=64)

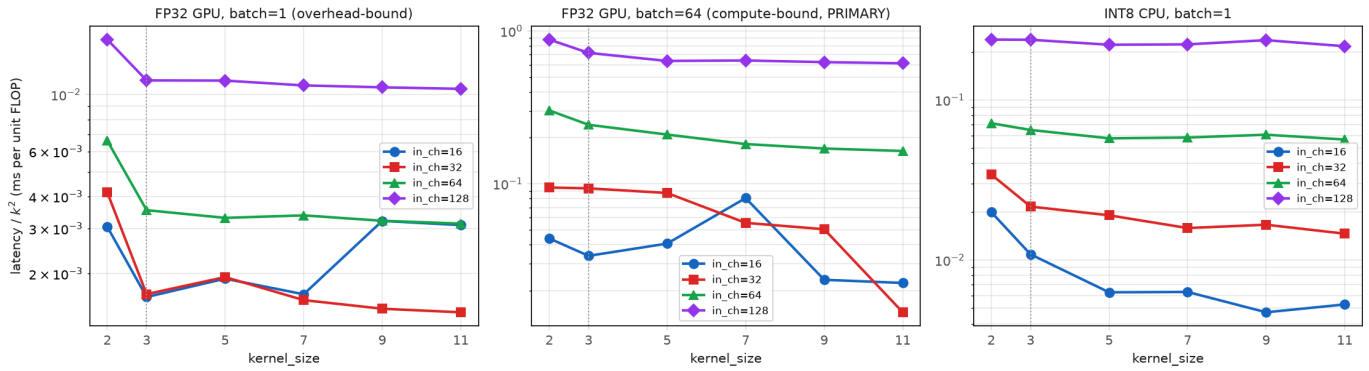


Figura 2. Latência por FLOP vs. tamanho de kernel (RTX 4060). O *scaling* sub-linear esperado de Winograd aparece de forma inconsistente entre regimes — ver texto para o teste estatístico e sua limitação no regime *compute-bound* (*batch*=64, painel central).

Accuracy vs. latency: Pareto frontier & baseline reference
(fp32 = GPU inference, int8 = CPU inference; per-precision deployment target)

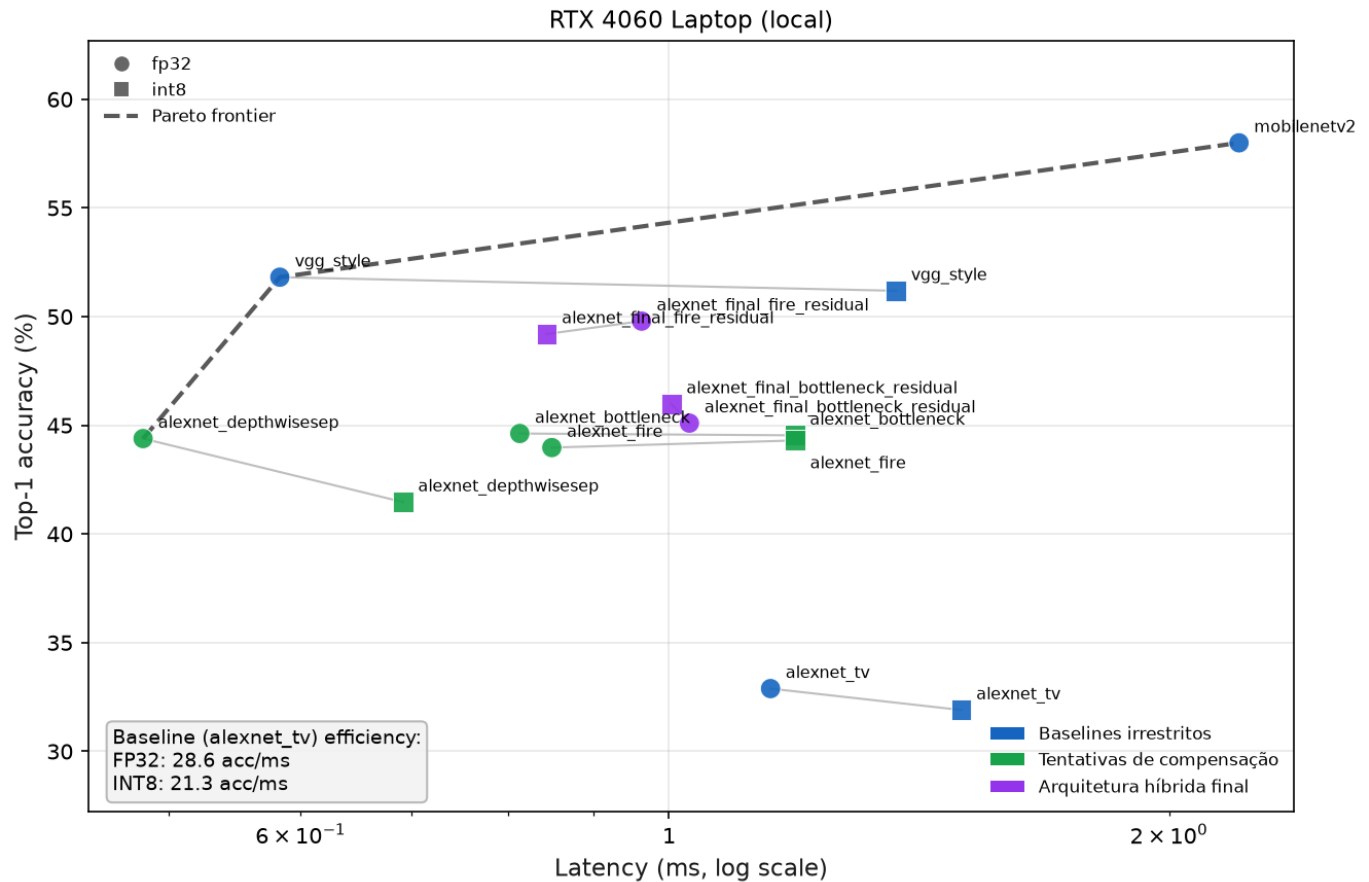


Figura 3. Fronteira de Pareto acurácia $\times$ latência (FP32 e INT8, RTX 4060, *batch*=1). Cor indica o mesmo agrupamento da Figura 1 (*baselines* irrestritos, restrição ingênua, tentativas de compensação, arquitetura híbrida final); forma indica precisão (círculo = FP32, quadrado = INT8). Modelos com convolução 3×3 densa (*bottleneck*, *Fire*) atingem latência competitiva com trade-off de acurácia face ao baseline irrestrito; modelos *depthwise* não recebem aceleração Winograd apesar do kernel pequeno.

AlexNetDepthwiseSep (+0,51 MB), ganha +10,75 p.p. de uma só vez, inclinação muito mais acentuada que qualquer

trecho interno da família *swin_pico*.

Sob INT8 quase nenhum sobrevive. A fronteira en-

Tabela III
ABLAÇÃO DO ATALHO RESIDUAL: ISOLANDO O COMPONENTE DECISIVO DA ARQUITETURA HÍBRIDA

Arquitetura	Params (M)	MACs (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	Ganho Vs. Fire (p.p.)
AlexNetFire (base)	0,52	177,7	43,98	44,30	—
AlexNetFireBypass	0,52	177,7	49,03	49,86	+5,05 FP32
AlexNetFinalFireResidual (híbrida completa)	0,70	55,2	49,79	49,20	+5,81 FP32

Acurácia vs. Tamanho — Fronteiras de Pareto (todos os modelos de classificação)

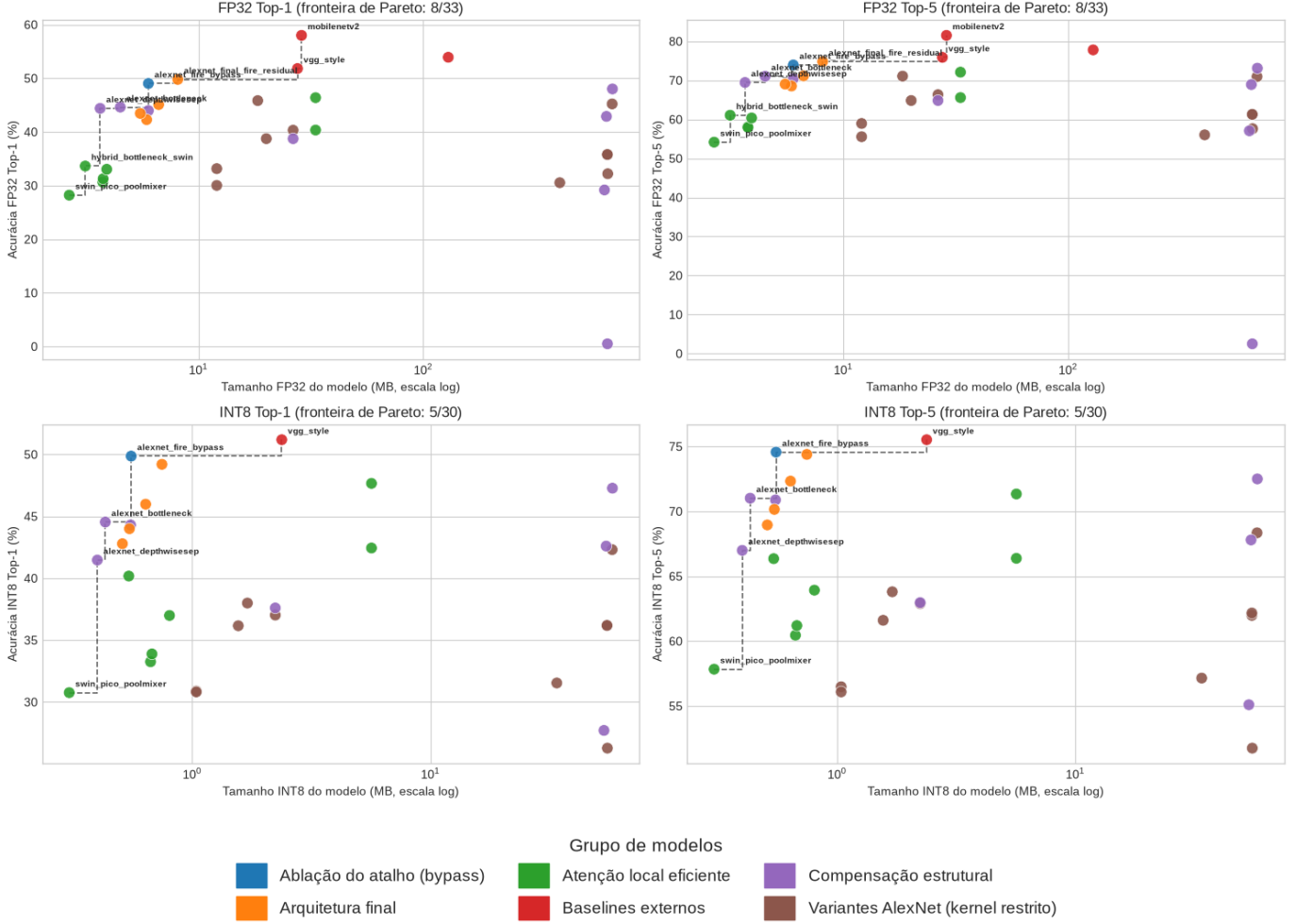


Figura 4. Acurácia vs. tamanho do *checkpoint* (MB, escala log) para os 33 modelos de classificação deste estudo (Fases 1–4, 8 e 9; o Eixo 6 mede mAP/mIoU, não top-1, e por isso não é comparável nesta mesma escala — omitida aqui). Quatro painéis: FP32 top-1/top-5 (superior) e INT8 top-1/top-5 (inferior), cada um com sua própria fronteira de Pareto tamanho-accurácia (linha tracejada; apenas os modelos na fronteira são rotulados, para legibilidade). Cor indica a fase de origem do modelo (mesma legenda em todos os painéis). O painel INT8 tem 5 de 30 modelos na fronteira (contra 8 de 33 em FP32) e apenas 30 dos 33 modelos aparecem — MobileNetV2 e ResNet18 pré-treinados nunca passaram por QAT (*residual adds* sem `FloatFunctional`, Seção III-A).

colhe para 5 modelos e só `swin_pico_poolmixer` (0,305 MB, 30,76%) permanece como ponto de atenção local; `hybrid_bottleneck_swin`, apesar do ganho incomum sob quantização (Eixo 7), é dominado por `AlexNetDepthwiseSep` (0,40 MB, 41,47%) — menor e mais acurado. Mesmo `deit_tiny` (46,38% FP32, o melhor modelo de atenção) fica dentro da fronteira, não sobre ela: seu *checkpoint* (~33,2 MB) é maior que o de VGG-Style (27,59

MB, 51,81%), que o domina em ambos os eixos.

O controle sem atenção é o pior modelo do estudo. `swin_pico_poolmixer` fica abaixo até da pior restrição ingênua da Fase 2 (`AlexNet2x2`, 30,02%) — excluindo `AlexNetSE`, cujo colapso de treino (~0,5% top-1) o torna um valor atípico, não um ponto de eficiência real. Isso é consistente com a hipótese de que a atenção em si, não apenas a arquitetura ao redor dela, contribui para a acurácia das

demais variantes `swin_pico`.

Os *baselines* pré-treinados perdem por tamanho, não por acurácia. MobileNetV2 e ResNet18 (sem ponto INT8) têm a melhor acurácia FP32 mas são muito maiores — uma restrição que desaparece num acelerador com foco em *throughput* em vez de latência pura.

Na Figura 5, a fronteira de Pareto FP32 tem só dois pontos: `swin_pico_poolmixer` (21,2M MACs, 28,21%), que entra apenas por ser o modelo de menor custo computacional do estudo — a mesma leitura já feita para o eixo de tamanho — e MobileNetV2 (25,8M MACs, 58,00%), que a domina de fato, com menos MACs que o próprio módulo *Bottleneck* (39,5M) e a melhor acurácia do estudo. Na fronteira INT8, mais povoada, os modelos de atenção não avançam além desse degrau inicial: `hybrid_bottleneck_swin` (30,3M MACs, 40,18%) fica de fora, dominado por `AlexNetFinalDepthwiseFire` (30,1M MACs, 42,79%) — menos MACs e mais acurácia.

O eixo MACs também reordena o agrupamento de compensação visível por tamanho na Figura 4: `AlexNetFire` e `AlexNetFireBypass` (177,7M MACs cada, já que o atalho por identidade não adiciona multiplicações) custam mais em computação que `AlexNetFinalFireResidual` (55,2M) e `AlexNetBottleneck` (39,5M) apesar de *checkpoints* bem menores — o mesmo padrão “poucos parâmetros, muitos MACs” do módulo *Fire* já discutido no Eixo 1.

E. Eixo 5: Robustez de Quantização e Comparação de Métodos de Compressão

Além de acurácia e latência, investigamos como diferentes mecanismos de compensação afetam a robustez sob quantização INT8 e compressão adicional.

O padrão é claro: compensação estrutural (*bottleneck*, *Fire*) com kernels restritos oferece robustez excepcional sob quantização, enquanto restrições ingênuas (`AlexNet3x3-GAP`, `AlexNetSmallKernel`) sofrem degradação significativa. `AlexNetSmallKernel` é a variante mais enxuta do estudo: mesmo *backbone* totalmente 3×3 do `AlexNet3x3-GAP`, mas com canais bem mais estreitos ($64\rightarrow 128\rightarrow 256\rightarrow 256\rightarrow 256$) e, como o `AlexNet3x3-GAP`, sem *BatchNorm*. É o modelo mais compacto do estudo em número de parâmetros dentre as variantes de kernel restrito sem compensação, mas também o que mais sofre com a quantização: sem BN após cada convolução e com canais estreitos, resta pouca margem de ativação para absorver o ruído de quantização — ao contrário de `AlexNetBottleneck/AlexNetFire`, que têm BN após toda convolução (Seção III-A). A Tabela IV agrupa modelos por família e mostra a queda de acurácia FP32 $\rightarrow$ INT8 (*drop*) para cada uma.

Com relação a métodos adicionais de compressão além de INT8 padrão, testamos precisão mista INT4/INT8 e investigamos *weight-sharing* pós-hoc. A precisão mista atribui bits por camada por sensibilidade: cada camada quantizável tem seus pesos fake-quantizados isoladamente para INT4 (por canal de

saída), mede-se o EQM dos *logits* resultantes contra o *baseline* FP32 num único *batch* de calibração, e as camadas são ranqueadas por essa sensibilidade — as 30% mais sensíveis permanecem em INT8, o restante vai para INT4, seguido de *fine-tuning*. A Figura 6 mostra que métodos sub-INT4 mais agressivos (*ternary*, *int2*, *binary*, todos QAT) sofrem quedas de 22 a 37 p.p., enquanto INT4 QAT permanece viável (<2 p.p. drop) para famílias de compensação, e essa precisão mista INT4/INT8 oferece a melhor taxa de compressão entre os métodos com retreino ($7,05\times$) mantendo acurácia média de 44,52%, um ganho médio de 0,19 p.p. sobre o FP32 dos mesmos 3 modelos.

Três ressalvas limitam a aplicabilidade imediata desses resultados: (i) INT4 PTQ sem retreino colapsa (25,3%) porque a calibração de quantização com um único *batch* não captura a distribuição real de ativações — o retreino (QAT) é crítico, triplicando a acurácia; (ii) INT4 em hardware real permanece fora do escopo: `fbgemm` (backend INT8 deste estudo) prioriza INT8, com suporte INT4 limitado; FPGA/aceleradores móveis suportam-no de forma inconsistente — esta medição é teórica; (iii) a compatibilidade com aceleração Winograd não é verificada — restrições de INT4 podem exigir reformatação de peso ou descontinuar a densidade que Winograd requer.

1) *Nota adicional: viabilidade de poda estruturada:* Como verificação adicional e preliminar (não um resultado central deste relatório), testamos poda estruturada de canais inteiros (não pesos individuais) no `AlexNetBottleneck`, razão 0,4, seleção por norma L1 [11]: 385.000 $\rightarrow$ 207.399 parâmetros ($-46,1\%$; 53,9% dos parâmetros originais retidos), mantendo toda convolução remanescente com `groups=1` (elegível à aceleração Winograd por construção). Sem *fine-tuning* após a poda a acurácia colapsa (top-1 0,50%), como esperado — este é apenas um teste de que a poda preserva `groups=1`, não um resultado de acurácia competitivo. A literatura de poda estruturada por norma L1 mostra que *fine-tuning* pós-poda recupera acurácia próxima à do modelo original [11], o que motiva tratar essa recuperação como trabalho futuro plausível, não apenas como esperança.

2) *Nota adicional: headroom de compressão via weight-sharing:* Em paralelo à poda, investigamos o potencial de compressão adicional além de INT8 puro, medindo quanto espaço ainda existe entre a entropia real dos pesos e a largura de bits nominal. Para o `AlexNetFire`, os pesos INT8 reais ocupam apenas 7,19 bits/peso de entropia de Shannon (contra 8,00 nominais), sugerindo margem para métodos de quantização com perda como *weight-sharing*. Aplicamos *k-means* clustering — o passo de codebook do Deep Compression [12] — em 16/32/64 *clusters* (correspondendo a 4/5/6 bits nominais), e observamos que o arquivo comprimido com *gzip* em disco fica 1,18 $\times$ menor que INT8 puro, chegando a $\sim 2,2\times$ de *headroom* adicional (só pesos) em 16 *clusters*. É uma medição teórica, não uma avaliação de acurácia real pós-clustering nem uma mudança efetiva no formato de checkpoint — apenas um sinal de que uma pipeline real de *weight-sharing* vale a pena construir (ver Trabalhos Futuros).

Acurácia vs. MACs: todos os modelos discutidos neste relatório (FP32 e INT8)

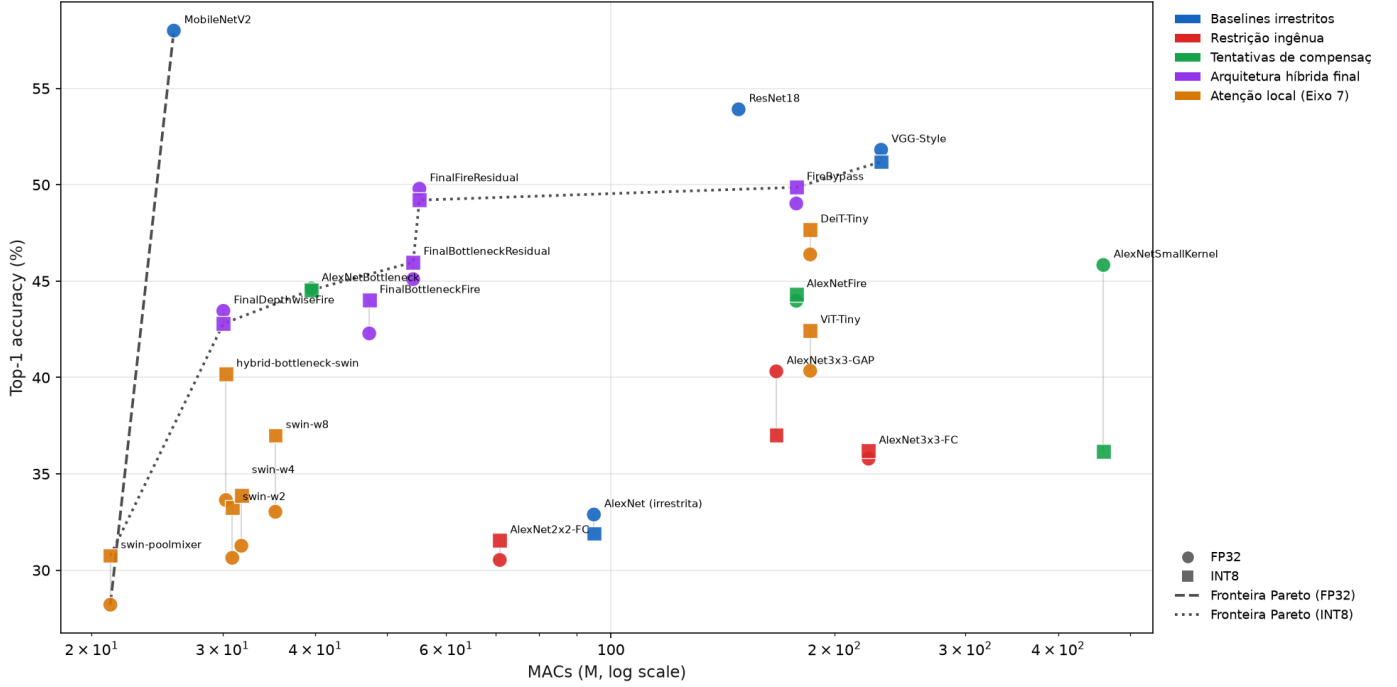


Figura 5. Acurácia vs. MACs (M, escala log) para 22 modelos — os 15 das Fases 1–4 e 9 mais os sete modelos de atenção local do Eixo 7 — do conjunto completo comparado por tamanho na Figura 4. MACs não muda entre FP32 e INT8 — mesma arquitetura, mesma contagem de multiplicação-acumulação — por isso os pontos FP32/INT8 de cada modelo compartilham a mesma posição no eixo X, unidos por uma linha vertical em vez da diagonal usada na Figura 4. Fronteira de Pareto tracejada (FP32) / pontilhada (INT8); um rótulo por modelo, posicionado sobre o ponto mais alto do par.

Tabela IV
ROBUSTEZ DE QUANTIZAÇÃO: QUEDA FP32→INT8 POR FAMÍLIA DE MODELO

Família	Modelo	Top-1 FP32	Top-1 INT8	Drop (p.p.)
Compensação com 3×3	AlexNetBottleneck	44,62%	44,54%	−0,08
	AlexNetFire	43,98%	44,30%	+0,33
Restrição ingênua	AlexNet3x3-GAP	40,32%	37,02%	−3,29
	AlexNetSmallKernel	45,84%	36,16%	−9,67
Baseline irrestrito	AlexNetTV	32,88%	31,90%	−0,98

F. Eixo 6: Transferência para Predição Densa

Os cinco eixos anteriores tratam exclusivamente de classificação. Esta seção pergunta se a compensação estrutural leve (*bottleneck*, *Fire*) e sua robustez à quantização observadas em classificação transferem para predição densa — detecção de objetos (SSD [17]) e segmentação semântica (DeepLabV3 [18]) — usando os três mesmos *backbones* (AlexNetBottleneck, AlexNetFire, AlexNetTV) sobre PASCAL VOC [19] (detecção: *trainval* 2007+2012, teste 2007; segmentação: *train/val* 2012). As métricas usadas são **mAP** (*mean Average Precision*) [19], a área média sob a curva precisão-recall por classe em detecção — resume acerto de localização (IoU entre caixa predita e caixa-verdade) e de classificação num único número —, e **mIoU** (*mean Intersection over Union*) [20], a sobreposição média entre máscara predita e máscara-verdade por classe em segmentação.

Uma investigação preliminar revelou recall de âncora

(fração de caixas-verdade cobertas por algum *anchor box* do *DefaultBoxGenerator*) de apenas 0,76–0,80 para os três *backbones*, bem abaixo do limiar de 95% considerado aceitável — causado por um erro de indexação nas camadas de extração de características (produzindo níveis de pirâmide duplicados) e por uma interpolação linear de escalas de âncora mal ajustada à distribuição real de tamanhos de caixa do VOC. Corrigidos os índices de *tap* e substituídas as escalas por valores explícitos ajustados a percentis da distribuição real, o recall sobe para 0,991/0,991/0,932 (*bottleneck/fire*/AlexNetTV) a 512px; os resultados abaixo usam exclusivamente execuções pós-correção. A quantização, em ambas as tarefas, incide apenas sobre o *backbone* — a *head* SSD/DeepLabV3 permanece em FP32.

As tabelas desta seção reportam três estágios, e a distinção entre os dois últimos importa para lê-las. A coluna **QAT** é o modelo em *quantization-aware training*: os pesos e ativações

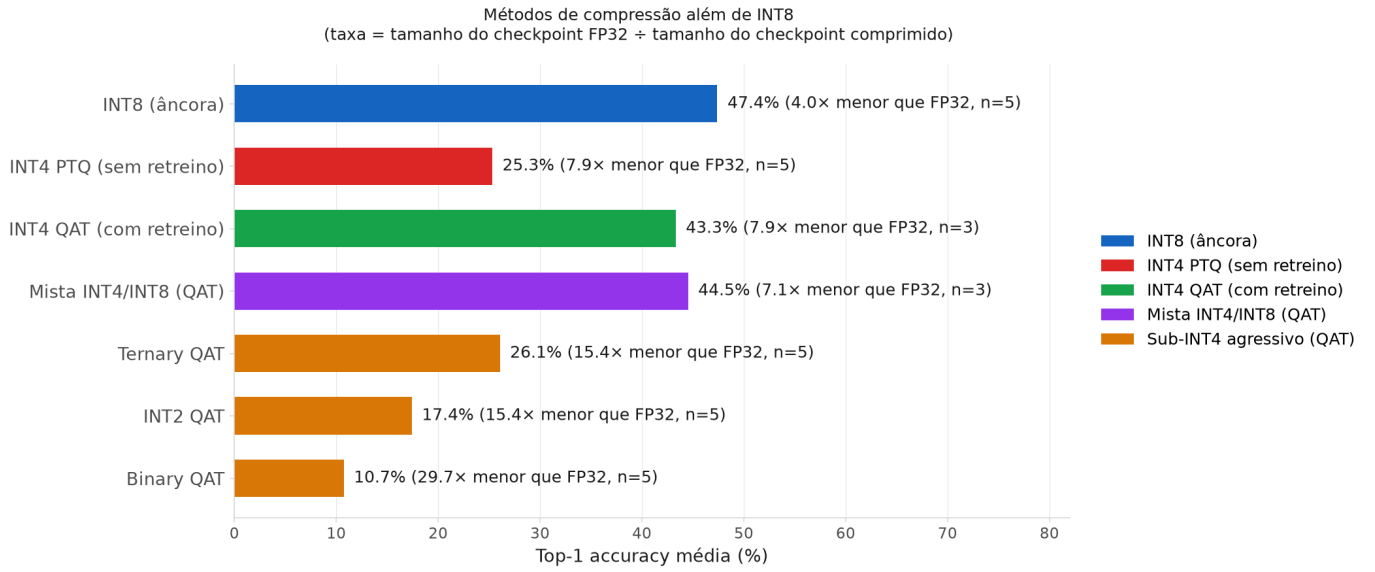


Figura 6. Acurácia top-1 média por método de compressão além de INT8, com taxa de compressão (tamanho do checkpoint FP32 dividido pelo tamanho do checkpoint comprimido) e número de modelos (n) entre parênteses. INT4 PTQ (sem retreino) colapsa para 25,3%; retreinar sob o mesmo INT4 (QAT) recupera para 43,3% — o retreino, não a largura de bits em si, é o que preserva a acurácia. Precisão mista INT4/INT8 (QAT) supera até o INT4 QAT uniforme com taxa de compressão comparável. Métodos sub-INT4 mais agressivos (ternary, int2, binary, todos QAT) degradam progressivamente.

Tabela V
EIXO 6 — DETECÇÃO (SSD, VOC, BACKBONE DO ZERO)

Backbone	mAP FP32	mAP QAT	mAP INT8
AlexNetBottleneck	20,98%	21,00%	20,85%
AlexNetFire	8,94%	6,38%	6,40%
AlexNetTV	16,94%	14,73%	14,50%

passam por *fake quantization* (arredondamento para a grade INT8 seguido de *dequantização*), mas a aritmética ainda é de ponto flutuante e a avaliação roda em GPU — é a acurácia que a quantização *simulada* prevê, medida no melhor *checkpoint* do *fine-tuning* de QAT. A coluna **INT8** é o modelo efetivamente convertido (*convert*, *backend* fbgemm): pesos e ativações passam a inteiros de 8 bits, os *kernels* são de fato inteiros e a inferência é obrigatoriamente em CPU, avaliada uma única vez sobre o mesmo conjunto de validação. A diferença QAT→INT8 isola, portanto, o erro residual da conversão real frente à simulação — pequena quando a calibração dos observadores é fiel, e o motivo de o valor INT8 poder ficar tanto ligeiramente abaixo quanto ligeiramente acima do QAT. Já a diferença FP32→INT8 é a medida de robustez à quantização usada no restante do relatório. A mesma convenção vale para as Tabelas V, VI e XII.

A Tabela V resume os resultados de detecção (variante treinada do zero; variantes com *backbone* pré-treinado em Tiny ImageNet-200 seguem padrão semelhante — ganham pouco para AlexNetBottleneck, mas recuperam parte da queda de QAT/INT8 para AlexNetFire e AlexNetTV). A Tabela VI resume segmentação.

Tabela VI
EIXO 6 — SEGMENTAÇÃO (DEEPLABV3, VOC 2012)

Backbone	mIoU FP32	mIoU QAT	mIoU INT8
AlexNetBottleneck	0,1784	0,1790	0,1791
AlexNetFire	0,1703	0,1712	0,1705
AlexNetTV	0,1634	0,1633	0,1621

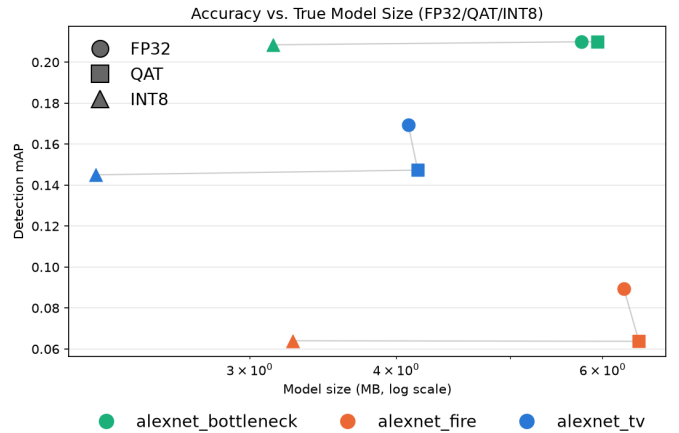


Figura 7. Detecção: acurácia (mAP) vs. tamanho real do modelo (*true_size_mb*, que exclui os pesos do classificador pré-treinado nunca usados no *forward pass* de detecção), FP32, QAT e INT8, para os três *backbones*.

A compensação de kernel pequeno transfere? Parcialmente, e o mecanismo importa. AlexNetBottleneck lidera detecção (mAP 20,98%) e mantém sobre o *backbone* de kernel irrestrito AlexNetTV (16,94%) a mesma vantagem que tinha em classificação. Já AlexNetFire, que pratica-

mente empata com AlexNetBottleneck em classificação (43,98% vs. 44,62% top-1, Eixo 1), cai para 8,94% — menos da metade. Paridade em classificação não implica paridade em localização.

A robustez de quantização transfere? Em segmentação, sim: a deriva de mIoU FP32→INT8 fica abaixo de 0,0013 nos três *backbones*, reproduzindo o achado “quantização quase não custa acurácia” do Eixo 5. Em detecção o quadro é mais ruidoso — AlexNetBottleneck permanece plano (−0,0014 mAP), enquanto AlexNetFire e AlexNetTV perdem ~0,024–0,025 mAP. AlexNetBottleneck é o único *backbone* cuja robustez se confirma sem ressalvas nas duas tarefas.

Predição densa é mais sensível a campo receptivo que classificação? Normalizando o mAP de detecção pela acurácia top-1 de classificação de cada *backbone* obtém-se 0,470 (*bottleneck*), 0,203 (*fire*) e 0,515 (AlexNetTV). AlexNetFire é o único caso de “pior em localização do que em classificação”; AlexNetTV, apesar de ser o classificador mais fraco dos três, é o menos penalizado pela troca de tarefa.

A head de detecção domina a latência? A latência do *pipeline* completo (*backbone*+*head* SSD, entrada 512×512) é 9,9×–15,0× maior que a do *backbone* isolado medida no Eixo 3 (entrada 64×64). A *head* e a resolução muito maior reintroduzem uma estrutura de latência que a comparação de kernel isolada no *backbone* não captura, diluindo qualquer sinal de aceleração Winograd no *pipeline* completo.

Um achado adicional e não previsto: o tamanho real do modelo (*true_size_mb*, que exclui pesos do classificador pré-treinado original nunca usados no *forward pass* de detecção ou segmentação) inverte o *ranking* de eficiência de tamanho aparente. O *checkpoint* bruto de AlexNetTV chega a 223 MB (a rede classificadora completa, majoritariamente não utilizada), mas seu tamanho real é de apenas 4,10 MB — menor que os 5,76 MB de AlexNetBottleneck — tornando-o o *backbone* mais eficiente em tamanho real nas duas tarefas, um resultado que o tamanho bruto do *checkpoint* escondia por completo.

Comparação com a literatura. Uma ressalva de métrica precede qualquer comparação: o mAP da Tabela V é a média sobre 10 limiares de IoU (0,50–0,95, convenção COCO — torchmetrics, chave “map”, ml/det_seg_trainer.py:276), enquanto toda a literatura de detecção citada aqui reporta mAP@IoU=0,5 de limiar único (convenção PASCAL VOC). O mesmo treino já calcula esse valor (chave “map_50”); é ele que a Tabela VII usa. Em segmentação não há discrepância: o mIoU (*macro* sobre 21 classes, ignore_index=255 para pixels de fronteira, ml/det_seg_trainer.py:320–322) já segue a convenção padrão da literatura.

Três diferenças de protocolo, essas sim, permanecem. Os *backbones* (0,39–0,52M parâmetros) são pré-treinados em Tiny ImageNet-200 (64×64, 200 classes), não em ImageNet completo. A segmentação usa o *split* original de VOC2012 (ml/det_seg_data.py, 1.464 imagens de treino) a 256px, sem o aumento via anotações SBD adotado por pratica-

Tabela VII

DETECÇÃO: EFICIÊNCIA ACURÁCIA/TAMANHO VS. LITERATURA (VOC, MAP@IoU=0,5 POR MB DE MODELO REAL — MESMA CONVENÇÃO DA LITERATURA CITADA; NOTE-SE QUE DIFERE DO MAP MÉDIO SOBRE IOU 0,5–0,95 DA TABELA V)

Modelo	Tam. (MB)	mAP@0,5	mAP@0,5/MB
SSD300 (VGG16) [17]	~105	77,2%	0,73
Fire SSD [27]	~26	70,7%	2,72
MobileNet-SSD [24]	22,0	68,0%	3,09
Pelee [25]	21,7	70,9%	3,26
Tiny-DSOD [26]	3,8	72,1%	18,97
AlexNetFire FP32	6,26	29,27%	4,68
AlexNetBottleneck FP32	5,76	47,46%	8,25
AlexNetTV FP32	4,10	37,67%	9,19
AlexNetFire INT8 [†]	~3,3	22,48%	~6,49
AlexNetTV INT8	2,22	33,09%	14,92
AlexNetBottleneck INT8	3,14	47,83%	15,23

Tabela VIII

SEGMENTAÇÃO: EFICIÊNCIA ACURÁCIA/TAMANHO VS. LITERATURA (VOC2012, mIoU POR MB DE MODELO REAL)

Modelo	Tam. (MB)	mIoU	mIoU/MB
FCN-8s (VGG16) [20]	~536	62,2%	0,116
DeepLabv3 (ResNet-101) [18]	~236	85,7%	0,363
DeepLabv3 + MobileNetV2 [6]	~18	70,0–75,7%	3,88–4,19
AlexNetFire FP32	12,63	17,03%	1,35
AlexNetBottleneck FP32	12,13	17,84%	1,47
AlexNetTV FP32	10,26	16,34%	1,59
AlexNetBottleneck INT8	11,19	17,91%	1,60
AlexNetTV INT8	9,32	16,21%	1,74

mente toda a literatura da área (~10.582 imagens, 480–513px de *crop*). E a detecção treina por 30 épocas sobre VOC07+12 *trainval*, uma fração do orçamento de ~240 épocas-equivalente do SSD original. As Tabelas VII–VIII comparam os três *backbones* com referências eficientes por acurácia por megabyte de modelo real implantado (*true_size_mb*) — eixo mais justo que a acurácia bruta, e mais alinhado à motivação de eficiência deste projeto.

Em detecção, os três *backbones* superam com folga os modelos densos clássicos, que nunca miraram tamanho de implantação: AlexNetBottleneck INT8 é ~21× mais eficiente que SSD300. Mais informativo é o confronto com os *designs* mobile-first, que perseguem exatamente esse objetivo — AlexNetBottleneck e AlexNetTV em INT8 superam MobileNet-SSD, Pelee e Fire SSD por 4,7–5,6×, e ficam a ~1,25× de Tiny-DSOD. O *design* dedicado ainda vence, mas por margem estreita.

Em segmentação a vantagem sobre os modelos densos se mantém (~15× sobre FCN-8s, ~4,8× sobre DeepLabv3/ResNet-101), mas a distância para os mobile-first é maior que em detecção: DeepLabv3+MobileNetV2 é ~2,2–2,4× mais eficiente em mIoU/MB que o melhor ponto deste estudo. A diferença de dado (VOC2012 sem aumento SBD, 256px) parece pesar mais nesta tarefa do que pesou em detecção. AlexNetFire é o pior dos três nas duas tarefas, reforçando o achado de que o mecanismo Fire não transfere

bem para predição densa.

Limitações deste eixo. Qualquer estatística de correlação aqui usa $n \leq 3$ *backbones* e tem poder estatístico desprezível — leia-se como descritivo, não como evidência. O recall de âncora de AlexNetTV (0,932) permanece abaixo da meta de 0,95, aceito como uma limitação estrutural de sua pirâmide de características nativamente mais grosseira, não perseguido adiante (a execução usa `--skip-anchor-check`). O *checkpoint* QAT/INT8 de AlexNetFire antecede uma correção do operador de concatenação quantizada do módulo *Fire*, então carrega sob o código atual apenas com `strict=False` (40 *buffers* de observador de quantização ausentes, todos escalares de poucos *bytes* cada — sem relação com peso de modelo real); com essa ressalva, *true_size_mb* foi recalculado normalmente para os dois estágios (Figura 7).

As comparações com a literatura carregam quatro ressalvas próprias. Os tamanhos de modelo das Tabelas VII–VIII são estimados a partir da contagem de parâmetros de cada artigo ($\text{params} \times 4$ bytes, FP32) quando o tamanho de *checkpoint* não é informado — aproximação, não tamanho de arquivo real. A linha marcada com † na Tabela VII usa o tamanho de *checkpoint* bruto, não *true_size_mb*, que não foi gravado naquela execução. O mIoU de 85,7% do DeepLabv3/ResNet-101 vem do *test set* oficial (via servidor de submissão), contra o *split val* usado aqui e por DeepLabv3+MobileNetV2 — são *splits* distintos, não diretamente comparáveis. E nenhuma referência mobile-first citada reporta variante INT8 no artigo original, então a comparação confronta o FP32 delas com o INT8 (mais compacto) destes modelos; em FP32 puro, AlexNetBottleneck/AlexNetTV ficam em 8,3–9,2 mAP@0,5/MB — ainda acima de MobileNet-SSD, Pelee e Fire SSD, e a $\sim 2\times$ de Tiny-DSOD.

G. Eixo 7: Atenção Local Eficiente

Esta seção pergunta se atenção local em janelas (*windowed self-attention*, ao estilo Swin [22]) reproduz o perfil de acurácia/eficiência/robustez-à-quantização das CNNs de kernel pequeno deste estudo, e se ela é, por construção, incompatível com aceleração Winograd independentemente do tamanho de janela.

Sete modelos testam essas hipóteses (Tabela IX). A família *swin_pico_{w2,w4,w8}* varre o tamanho de janela $\in \{2, 4, 8\}$ — o análogo em atenção da restrição de kernel do Eixo 1. *hybrid_bottleneck_swin* antepõe um *stem* CNN de compensação aos estágios de atenção. *swin_pico_poolmixer* troca a atenção por um *pooling mixer* simples, controlando se o ganho vem da atenção real ou apenas da arquitetura ao redor dela. E o par *vit_tiny/deit_tiny* usa atenção global ao estilo ViT [21], com e sem destilação DeiT [23] de um professor MobileNetV2 congelado.

Como não há caminho INT8 estável para LayerNorm/*softmax*/atenção neste *pipeline eager-mode* de QAT baseado em fbgemm, essas camadas permanecem em FP32 na conversão — apenas Linear/Conv vão para INT8, mesma exclusão seletiva já usada para BatchNorm neste estudo.

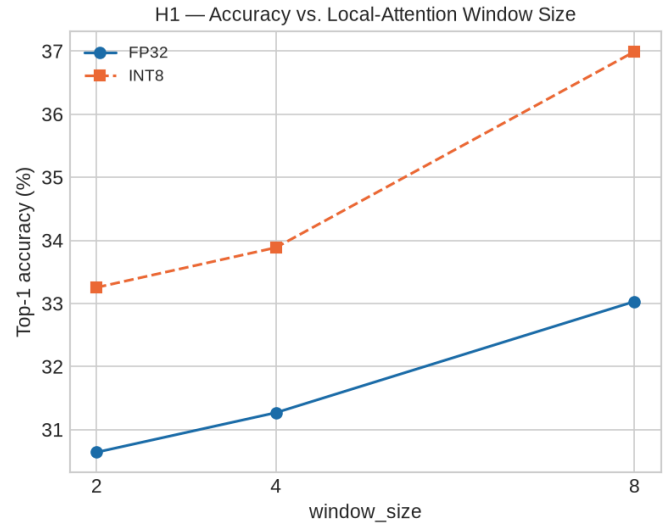


Figura 8. Acurácia top-1 (FP32 e INT8) vs. tamanho de janela de atenção local $\in \{2, 4, 8\}$ — análogo, em atenção, da curva de restrição de kernel do Eixo 1.

O tamanho de janela reproduz a curva de restrição de kernel? Sim. A acurácia top-1 cresce monotonicamente com a janela tanto em FP32 (30,64%→31,27%→33,03% para janelas 2/4/8) quanto em INT8 (33,25%→33,89%→36,99%): restringir o campo receptivo por camada custa acurácia também neste paradigma.

O stem CNN ajuda? Frente à atenção pura, sim: o híbrido supera a Swin de janela máxima (*swin_pico_w8*) por +3,19 p.p. em INT8 a tamanho semelhante (0,542 vs. 0,803 MB). Mas não fecha a distância até as CNNs de compensação a tamanho equivalente — AlexNetFireBypass ocupa quase o mesmo espaço INT8 ($\sim 0,55$ MB) e atinge 49,86% contra 40,18% do híbrido.

A atenção é menos robusta à quantização? Este é o resultado mais contra-intuitivo do eixo: é o contrário. Em vez de perderem acurácia sob INT8 (hipótese original, motivada pela exclusão de LayerNorm/atenção do QAT), os sete modelos *ganham* — de +1,28 a +6,54 p.p. (coluna Δ QAT, Tabela IX), acima dos ganhos já notáveis das próprias CNNs de compensação (AlexNetFire +0,33 p.p., AlexNetFireBypass +0,84 p.p.). Uma leitura plausível é que, ao manter LayerNorm/atenção em FP32 e converter apenas as camadas Linear/Conv, o *fine-tuning* de QAT funciona como regularização adicional sobre a fração convertida do modelo — mas o mecanismo exato permanece especulativo.

A destilação DeiT compensa? Confirma a hipótese: *deit_tiny* supera *vit_tiny* (mesma arquitetura, mesmos hiperparâmetros, só a função de perda muda) por +6,04 p.p. em FP32 e +5,22 p.p. em INT8. O ganho de destilar de um professor MobileNetV2 congelado sobrevive à quantização e faz de *deit_tiny* o modelo mais forte deste eixo.

Algum modelo de atenção aciona Winograd? Nenhum, em nenhuma configuração testada (*winograd_trace_detected* = falso em todos). O

Tabela IX
EIXO 7 — MODELOS DE ATENÇÃO LOCAL: PARÂMETROS E ACURÁCIA

Modelo	Params (M)	MACs (M)	Top-1 FP32 (%)	Top-1 INT8 (%)	ΔQAT (p.p.)
swin_pico_w2 (janela 2)	0,32	30,9	30,64	33,25	+2,61
swin_pico_w4 (janela 4)	0,32	31,8	31,27	33,89	+2,62
swin_pico_w8 (janela 8)	0,32	35,3	33,03	36,99	+3,96
swin_pico_poolmixer	0,23	21,2	28,21	30,76	+2,55
hybrid_bottleneck_swin	0,27	30,3	33,64	40,18	+6,54
vit_tiny	2,76	185,5	40,35	42,44	+2,10
deit_tiny	2,76	185,5	46,38	47,66	+1,28

resultado, porém, já era previsível pela arquitetura: nenhum dos sete modelos contém convolução 3×3 de *stride* 1 — a única estrutura que aciona Winograd $F(2 \times 2, 3 \times 3)$. As únicas convoluções presentes são os *patch embeddings* (4×4 em swin_pico, 8×8 em vit_tiny/deit_tiny, ambos com *stride* igual ao *kernel*) e, no híbrido, um *stem bottleneck* de *stride* 2. A medição confirma a expectativa, mas não isola a atenção como causa: CNNs puras sem 3×3 de *stride* 1 — AlexNetBottleneck e ResNet18, no próprio Eixo 3 — também não acionam Winograd. Ao contrário do achado sobre convoluções *depthwise*, aqui não há contrafactual medido.

Uma ressalva central atravessa todo este eixo: vit_tiny/deit_tiny carregam 2,76M parâmetros — cerca de $9 \times$ mais que a família swin_pico (0,23–0,32M) e muito próximo, em escala, de VGG-Style (2,41M, Eixo 1) — não por acaso: VGG-Style é o modelo mais próximo em tamanho de todo o projeto, servindo de referência natural — mas $5\text{--}7 \times$ maior que as CNNs de compensação Pareto-ótimas (AlexNetBottleneck 0,39M, AlexNetFire/AlexNetFireBypass 0,52M) com as quais competem em acurácia. O resultado de destaque de deit_tiny (46–48% top-1) deve ser lido contra esse tamanho — competitivo com VGG-Style (51,81% FP32) a tamanho semelhante, mas não gratuito frente às CNNs de compensação.

Limitações deste eixo. A verificação de compatibilidade com Winograd mede apenas o modelo inteiro — não há, no código de perfilamento atual, discriminação por submódulo (*stem* vs. estágio de atenção) do tempo de dispositivo; a mesma ressalva do Eixo 3 sobre a instabilidade do nome do *kernel* cuDNN entre versões se aplica aqui, tornando a ausência de traço Winograd uma evidência fraca, não uma prova. A razão INT8/FP32 de tamanho reportada acima para a robustez de quantização usa o tamanho de *checkpoint* em disco, não a contagem de parâmetros ponderada por precisão por módulo proposta originalmente para essa hipótese — uma proxy razoável de tamanho de implantação, mas não a mesma grandeza.

H. Síntese dos sete eixos

Os sete pilares deste trabalho conectam-se em uma narrativa coerente:

- 1) *Custo da Restrição de Kernel (Eixo 1)*: restringir kernels a 3×3 isoladamente não custou acurácia neste protocolo;

a troca de *head* para GAP foi a mudança com efeito isolado mais claro (+4,5 p.p.).

- 2) *Mecanismos de Compensação (Eixo 1)*: *bottleneck* e *Fire* recuperam mais 3,7–4,3 p.p. com parâmetros drasticamente reduzidos, mas não reduzem MACs na mesma proporção.
- 3) *Sinergia Híbrida (Eixo 2)*: Fire + Residual rende mais que a soma de seus efeitos (+5,81 p.p. acima do Fire de base); nem toda combinação sinergiza igualmente bem.
- 4) *Validação em Hardware (Eixo 3)*: o cuDNN aciona Winograd para convoluções densas 3×3 em *batches* pequenos, nunca para *depthwise*; no regime *compute-bound*, *tensor cores* parecem competir com a vantagem de Winograd.
- 5) *Atalho de Custo Zero (Eixo 4)*: um atalho residual isolado fecha 87% do ganho da arquitetura híbrida completa, sem parâmetros extras.
- 6) *Transferência para Predição Densa (Eixo 6)*: a compensação estrutural transfere para detecção e segmentação de forma específica por mecanismo, não universal — AlexNetBottleneck mantém sua vantagem, AlexNetFire colapsa.
- 7) *Atenção Local Eficiente (Eixo 7)*: atenção em janelas reproduz a curva de restrição de *kernel* e não aciona Winograd — por ausência de convoluções elegíveis, não por um efeito da atenção em si; contra a expectativa inicial, os sete modelos *ganham* acurácia sob INT8.

IV. DISCUSSÃO

Os modelos com compensação (*bottleneck*, *Fire*) mostram robustez excepcional sob quantização INT8 (−0,08 a +0,33 p.p.), enquanto a restrição ingênua sofre 3–10 p.p. de queda. Uma hipótese plausível é que as convoluções 1×1 do *bottleneck/squeeze* produzem ativações com distribuições mais regulares, facilitando a calibração de quantização; um teste direto dessa hipótese permanece trabalho futuro.

No plano de hardware, medições em RTX 4060 mostram que *tensor cores* competem com Winograd em *batches* grandes (≥ 64) — uma limitação da plataforma de validação, não do princípio em si (Eixo 3 detalha a medição e suas ressalvas). Na mesma linha, MobileNetV2 e AlexNetDepthwiseSep, apesar de terem kernels pequenos, nunca acionam um *kernel* Winograd nesta GPU/backend ($\sim 51\%$ menos GFLOP/s de mediana que modelos densos comparáveis) — o que sugere que arquiteturas eficientes para dispositivos móveis podem necessitar

estratégias de otimização diferentes daquelas propostas aqui, sem que isso implique incompatibilidade matemática entre Winograd e convoluções *depthwise* em geral.

Limitações. (i) A comparação com a baseline irrestrita (Tabela I) mistura três fatores — tamanho de kernel, *head* FC/GAP e pré-treino ImageNet — que não são completamente separáveis com os controles deste estudo; AlexNet3×3-FC isola o primeiro dos dois últimos, mas nenhuma variante restrita é pré-treinada, então o efeito do pré-treino especificamente permanece confundido. (ii) O modelo `alexnet_tv` foi treinado de forma independente em dois momentos do projeto (79 e 67 épocas), com resultados diferentes, especialmente em INT8 (31,90% vs. 26,28%); este relatório padroniza na execução de 79 épocas (a mesma usada no perfilamento de hardware) em todas as tabelas, mas a diferença entre execuções é, em si, um indício de variância não desprezível entre treinos nominalmente idênticos. (iii) Toda a validação de hardware é de uma única GPU (RTX 4060 Laptop), em uma única sessão de coleta — sem repetição entre sessões, sem outra GPU, e sem qualquer medição em FPGA; o Eixo 3 já discute como isso limita a leitura dos testes estatísticos de latência. (iv) Os testes estatísticos de hardware operam com n pequeno (8–24 configurações), o que limita seu poder — a evidência mais confiável desta seção é o rastreamento direto do nome do *kernel*, não os testes de latência. (v) Nenhum dos resultados de acurácia usa múltiplas sementes de treino; diferenças de poucos décimos de ponto percentual entre arquiteturas próximas não devem ser lidas como significativas.

V. CONCLUSÃO

Restringir kernels de CNNs a 3×3 para compatibilidade com aceleradores Winograd não se mostrou claramente custoso em acurácia neste protocolo, quando isolado de outras mudanças (Eixo 1) — um resultado que contraria a expectativa inicial deste projeto. O que de fato eleva a acurácia acima da baseline pré-treinada é a combinação com um *head* GAP e mecanismos de compensação leves — blocos *bottleneck*, módulos *Fire*, ou atalhos residuais isolados — nenhum dos quais reintroduz kernel $> 3 \times 3$.

Com apenas 0,385M parâmetros (4,49 MB) e 39,5M MACs — menos de um quarto dos MACs do módulo *Fire* (177,7M), a menor contagem de computação entre os mecanismos de compensação testados — o AlexNetBottleneck atinge 44,62% de acurácia top-1 em FP32 e mantém 44,54% após INT8 (drop de apenas 0,08 p.p., o melhor do estudo). Entre os mecanismos de compensação estrutural testados, é o único que reduz memória e computação simultaneamente frente à baseline restrita+GAP, relevante para um acelerador cujo custo escala principalmente com número de multiplicações. A ablação por atalho residual isolado revela que essa adição sozinha (zero parâmetros extras) captura aproximadamente 87% do ganho de uma arquitetura híbrida muito mais completa (5,05 de 5,81 p.p.) nesta família de arquiteturas, sugerindo que simplicidade estrutural pode aproximar-se de elaboração arquitetural complexa, ao menos aqui — com a ressalva crítica de que o AlexNetFireBypass usou um orçamento de

épocas maior que o AlexNetFinalFireResidual (152 vs. 77 épocas, Apêndice B), então parte desse ganho pode refletir o treino mais longo, não apenas a mudança arquitetural.

A validação em hardware (RTX 4060, sessão única) mostra, por rastreamento direto do nome do *kernel* CUDA, que o cuDNN aciona um *kernel* Winograd para convoluções densas 3×3 em *batches* pequenos (1 e 8) — e nunca para convoluções *depthwise*, em nenhum tamanho de kernel testado. O teste estatístico de *scaling* de latência é, no entanto, significativo na direção oposta à esperada no regime *compute-bound* (*batch*=64), o mais relevante para um acelerador dedicado. Três achados limitam o escopo desta validação: (i) *tensor cores* parecem competir com a vantagem de Winograd justamente no regime *compute-bound*, tornando o ganho de latência inconsistente entre *batches*; (ii) convoluções *depthwise* nunca acionam Winograd nesta GPU/backend, ajudando a explicar por que MobileNetV2 não se beneficia como as arquiteturas densas; (iii) toda a medição vem de uma única GPU e uma única sessão de coleta. Um acelerador FPGA dedicado (motivação deste projeto) não possui *tensor cores* e, por construção, não sofreria a competição de (i) — mas essa é uma expectativa de *design*, não algo medido neste trabalho.

Estendendo essas descobertas além de classificação, a transferência para predição densa (Eixo 6) mostra que o benefício da compensação estrutural não é universal: mantém-se integralmente para AlexNetBottleneck em detecção e segmentação, mas colapsa para AlexNetFire apesar de os dois mecanismos serem quase intercambiáveis em classificação — qual mecanismo específico foi escolhido importa mais para predição densa do que para classificação. Já a investigação de atenção local (Eixo 7) confirma a expectativa estrutural do princípio deste projeto — nenhum dos sete modelos testados aciona Winograd em qualquer configuração, por ausência de convoluções elegíveis e não por um efeito da atenção em si, e a acurácia segue a mesma curva de restrição de campo receptivo observada para kernels convolucionais — mas produz um resultado de quantização oposto ao esperado: em vez de menos robustos, os modelos de atenção local *ganham* acurácia sob INT8 mais que as próprias CNNs de compensação, plausivelmente porque manter LayerNorm/atenção em FP32 durante o QAT (evitando um caminho de quantização instável) transforma o restante do *fine-tuning* em regularização adicional. A destilação DeiT é o modelo de atenção mais forte deste estudo (46,38% FP32), mas a 5–7× o número de parâmetros das CNNs Pareto-ótimas deste projeto (2,76M vs. 0,39–0,52M) — competitivo em acurácia, não em eficiência.

VI. TRABALHOS FUTUROS

- 1) *Fine-tuning* pós-poda estruturada: Recuperar acurácia do AlexNetBottleneck podado (46,1% menos parâmetros, 0,385M → 0,207M), mantendo Winograd-elegibilidade via `groups=1` — literatura de poda por norma L1 [11] sugere que essa recuperação é factível.
- 2) *Weight-sharing* QAT-aware: Implementar pipeline real de *weight-sharing* [12] com retreino (contraste com a

medição teórica de *headroom* já feita neste estudo), explorando os 7,19 bits/peso efetivos já usados por INT8.

- 3) Hardware profiling estendido: Medições de latência e potência em FPGA real ou acelerador dedicado, validando hipótese de tensor cores ser artefato de GPU.

As investigações centrais de transferência para predição densa (Eixo 6 — detecção e segmentação) e de atenção local eficiente (Eixo 7) já estão concluídas, com resultados completos apresentados neste relatório (Seções III-F e III-G); as lacunas pontuais remanescentes de cada uma estão descritas em suas respectivas “Limitações deste eixo”. Esses dois eixos confirmam que o framework de kernel-restriction + compensation se estende, ainda que de forma específica por mecanismo, para além de classificação (predição densa) e além de CNNs puras (atenção local).

AGRADECIMENTOS

Experimentos utilizaram recursos da infraestrutura PCAD (<http://pcad.inf.ufrgs.br>).

REFERÊNCIAS

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.
- [2] A. Lavin and S. Gray, “Fast algorithms for convolutional neural networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [3] B. Barabasz, A. Anderson, K. M. Soodhalter, and D. Gregg, “Error analysis and improving the accuracy of Winograd convolution for deep neural networks,” *ACM Transactions on Mathematical Software*, vol. 46, no. 4, Art. 37, 2020.
- [4] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, “SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size,” *arXiv preprint arXiv:1602.07360*, 2016.
- [5] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [7] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [8] D. Khudia, J. Huang, P. Basu, S. Deng, H. Liu, J. Park, and M. Smelyanskiy, “FBGEMM: Enabling high-performance low-precision deep learning inference,” *arXiv preprint arXiv:2101.05615*, 2021.
- [9] Y. Le and X. Yang, “Tiny ImageNet visual recognition challenge,” Stanford CS231N Report, 2015.
- [10] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [11] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, “Pruning filters for efficient ConvNets,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [12] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” in *International Conference on Learning Representations (ICLR)*, 2016.
- [13] G. Li, R. Li, T. Li, T. Chen, M. Zhang, and H. Corporaal, “Algorithm-hardware co-design for accelerating depthwise separable CNNs,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 30, no. 2, 2025.
- [14] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [15] E. D. Cubuk, B. Zoph, D. Mané, V. Vasudevan, and Q. V. Le, “AutoAugment: Learning augmentation strategies from data,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 113–123.
- [16] M. Lin, Q. Chen, and S. Yan, “Network in network,” in *International Conference on Learning Representations (ICLR)*, 2014.
- [17] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single shot multibox detector,” in *Proc. European Conference on Computer Vision (ECCV)*, 2016.
- [18] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, “Rethinking atrous convolution for semantic image segmentation,” *arXiv preprint arXiv:1706.05587*, 2017.
- [19] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.
- [20] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.
- [21] A. Dosovitskiy *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [22] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin Transformer: Hierarchical vision transformer using shifted windows,” in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [23] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” in *Proc. International Conference on Machine Learning (ICML)*, 2021.
- [24] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” *arXiv preprint arXiv:1704.04861*, 2017.
- [25] R. J. Wang, X. Li, and C. X. Ling, “Pelee: A real-time object detection system on mobile devices,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [26] Y. Li, J. Li, W. Lin, and J. Li, “Tiny-DSOD: Lightweight object detection for resource-restricted usages,” in *Proc. British Machine Vision Conference (BMVC)*, 2018.
- [27] H. Liao, N. Yamini, and Y. Wong, “Fire SSD: Wide fire modules based single shot detector on edge device,” *arXiv preprint arXiv:1806.05363*, 2018.

A. Diagramas de Arquitetura

Para os quatro modelos que sustentam os Eixos 1, 2 e 4 — os que ocupam a fronteira de Pareto acurácia/tamanho/quantização e são construídos inteiramente com convoluções $1 \times 1/3 \times 3$ (nenhum kernel $> 3 \times 3$) — as Figuras 9–12 mostram o diagrama de blocos por estágio, com canais de entrada/saída, contagem de parâmetros e o tipo de cada convolução interna (1×1 em cinza, 3×3 em verde), extraídos dos pesos treinados reais. Os diagramas são dispostos na horizontal (entrada à esquerda, *head* classificador à direita) para caber dois por página em largura total sem perder legibilidade. Os nomes de camadas do PyTorch (Conv2d, Linear, MaxPool2d, GAP) e os termos *squeeze/expand* são mantidos como na implementação.

B. Dados de Treinamento

A Tabela X resume o protocolo de hiperparâmetros compartilhado por todos os modelos (`configs/training.yaml`, `qat.yaml`, `data.yaml`); a taxa de aprendizado inicial é um hiperparâmetro por modelo, listada na Tabela XI. No QAT, a BatchNorm é congelada na época 3 e os observadores de quantização desativados na época 5, seguindo a prática recomendada por [7].

A Tabela XI consolida, por modelo, taxa de aprendizado, épocas e tempo de treino por estágio (FP32/QAT) e acurácia final (FP32/INT8), agrupados como na Figura 1. Células “—” de QAT valem para modelos anteriores ao Eixo 7, cujos *checkpoints/logs* de QAT não foram preservados; INT8 não é um estágio treinado (calibração estática em segundos) e por isso não tem coluna de tempo. *Baselines* irrestritos, restrição ingênua, AlexNetBottleneck e AlexNetFire treinaram localmente (RTX 4060 Laptop); as híbridas do Eixo 2, a arquitetura híbrida final e os sete modelos do Eixo 7 treinaram no cluster PCAD (RTX 4090) — tempos entre os dois ambientes, e frente à latência do Eixo 3 (RTX 4060, Seção II-C), não são comparáveis. O AlexNetFireBypass usou um orçamento de épocas maior que os demais por rodar no PCAD (152 vs. ~ 40 –77), daí seu tempo desproporcional. Os modelos do Eixo 7 compartilham uma única taxa de aprendizado/*weight decay* ($5 \times 10^{-4}/0,05$, receita de [23]) em vez do valor por modelo das demais famílias; a maioria treinou via `scripts/train.py` e encerrou por *early stopping* (paciência 50) bem antes do teto de época, exceto `vit_tiny/deit_tiny`, via *notebook* dedicado (Seção III-G).

C. Dados de Treinamento — Predição Densa (Eixo 6)

A Tabela XII lista, por *backbone*, tarefa e variante (treinada do zero ou com *backbone* pré-treinado no Tiny ImageNet-200, Seção III-F), a época do melhor *checkpoint*, o tempo de treino FP32 e a métrica FP32 correspondente (mAP para detecção, já apresentada na Tabela V apenas para a variante do zero; mIoU para segmentação, Tabela VI), todos executados nos nós RTX 4090 do cluster PCAD (partição `tupi`). Segmentação não tem variante pré-treinada (`--pretrained-ckpt` não

é suportado para essa tarefa, Seção III-F). O orçamento de detecção foi ampliado para até 1000 épocas com paciência de *early stopping* 50 (`docs/PHASE7_LOG.md`, Estágio 9) após a correção do *bug* de recall de âncora — daí os tempos de detecção, da ordem de dias, bem mais longos que os de classificação (Tabela XI) ou segmentação, cujo *early stopping* interrompeu muito antes desse teto. As colunas de métrica cobrem os três estágios (FP32/QAT/INT8); os valores das variantes do zero repetem os das Tabelas V e VI, e as variantes pré-treinadas de detecção aparecem aqui pela primeira vez em QAT/INT8. O tempo tabulado é o do estágio FP32 apenas: QAT tem orçamento de época próprio e mais curto, e INT8 não é um estágio treinado. Partir de um *backbone* pré-treinado reduz o tempo de treino de detecção para AlexNetBottleneck (56,1h vs. 29,7h do zero — mais épocas até o melhor *checkpoint*, 709 vs. 549, mas cada uma mais barata) e para AlexNetTV (125,1h vs. 115,0h), mas não para AlexNetFire (88,4h vs. 68,6h, mais lento mesmo pré-treinado). As épocas e tempos dos estágios QAT e INT8 não são tabulados por brevidade; ver Seção III-F para a discussão das métricas de cada estágio.

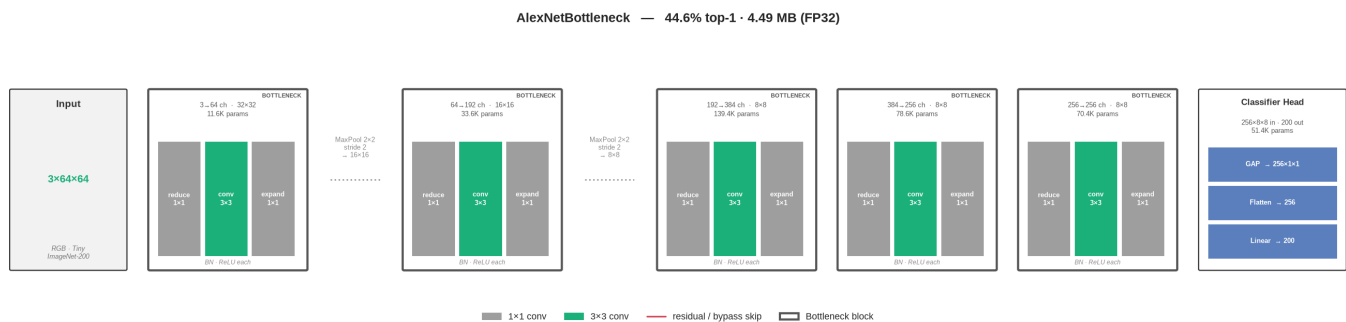


Figura 9. AlexNetBottleneck: bloco *reduce* $1 \times 1 \rightarrow \text{conv } 3 \times 3 \rightarrow \text{expand } 1 \times 1$ por estágio.

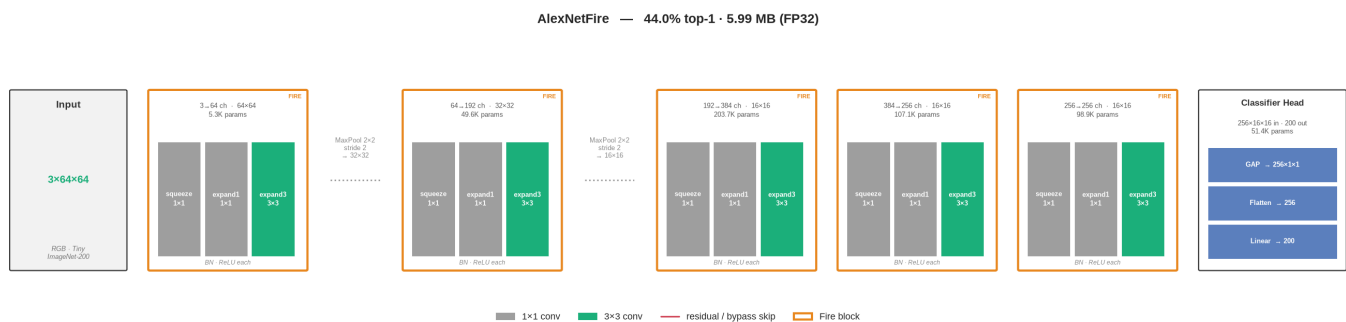


Figura 10. AlexNetFire: módulo *Fire* (*squeeze* $1 \times 1 \rightarrow \text{expand } 1 \times 1 / 3 \times 3$ concatenados) por estágio.

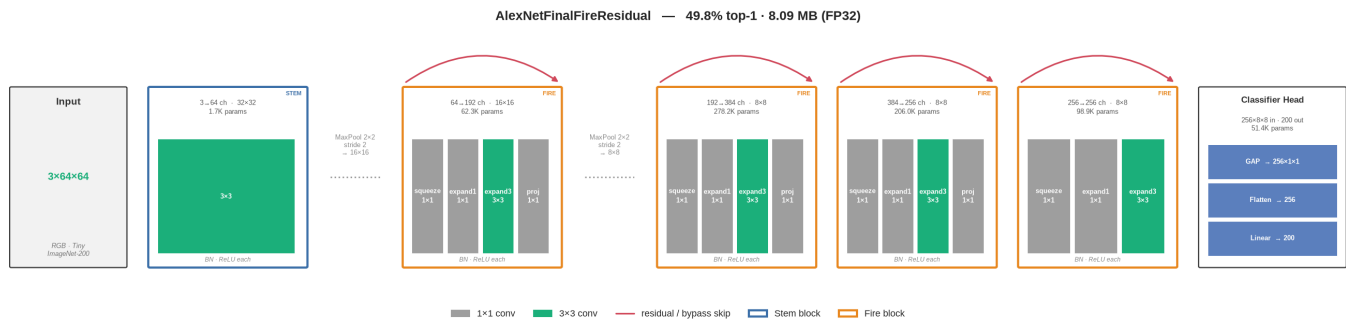


Figura 11. AlexNetFinalFireResidual: *stem* 3×3 *stride*-2 seguido de módulos *Fire* com atalho residual em cada estágio.

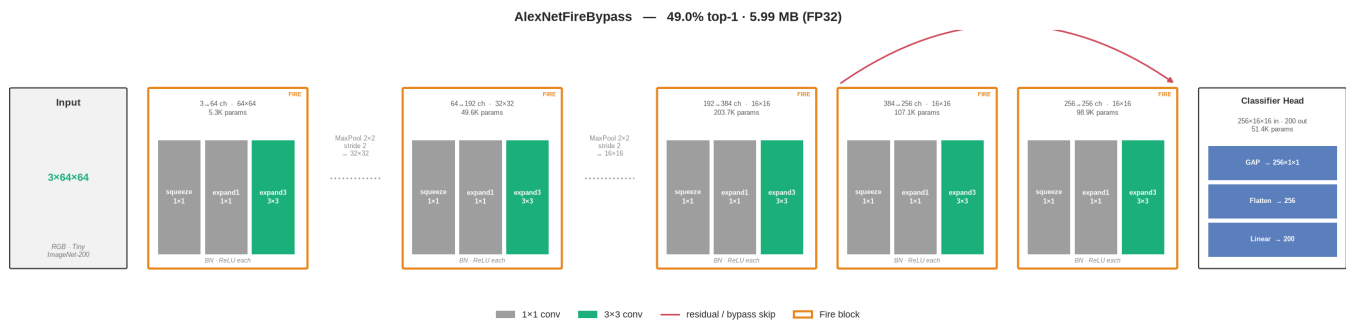


Figura 12. AlexNetFireBypass: AlexNetFire base com um atalho residual por identidade adicionado a cada estágio, sem nenhum parâmetro extra.

Tabela X
HIPERPARÂMETROS DE TREINAMENTO COMPARTILHADOS

Parâmetro	Valor
Otimizador	AdamW [14]
<i>Scheduler</i>	<i>Cosine annealing</i> (T_max = épocas)
Épocas (orçamento FP32)	100, <i>early stopping</i> paciência 5
Épocas (orçamento QAT)	20, <i>early stopping</i> paciência 5
<i>Weight decay</i> (FP32 / QAT)	5×10^{-4} / 5×10^{-4}
<i>Label smoothing</i>	0,1
<i>Batch size</i>	64
AMP (<i>mixed precision</i>)	Ligado (FP32) / desligado (QAT)
<i>Split</i> treino/validação	90/10 determinístico, seed 42
Aumento de dados (treino)	<i>RandomResizedCrop</i> (0,7–1,0), <i>flip</i> horizontal, <i>RandomRotation</i> (15°), <i>AutoAugment</i> (política ImageNet) [15]
Aumento de dados (validação)	<i>Resize</i> → <i>CenterCrop</i>
Normalização	Média/desvio-padrão do ImageNet
QAT: <i>freeze</i> BN / desativa <i>observers</i>	Época 3 / Época 5
INT8: <i>backend</i> / dispositivo	fbgemm / CPU

Tabela XI
TAXA DE APRENDIZADO, ÉPOCAS EFETIVAS (FP32/QAT), TEMPO DE TREINO (FP32/QAT) E ACURÁCIA TOP-1 (FP32/INT8) POR MODELO

Família	Modelo	LR	Ép. FP32	Ép. QAT	T. FP32 (min)	T. QAT (min)	Top-1 FP32 (%)	Top-1 INT8 (%)
Baselines irrestritos	AlexNetTV (pré-trein.)	3×10^{-4}	79	—	63,2	—	32,88	31,90
	MobileNetV2 (pré-trein.)	1×10^{-4}	27	—	14,8	—	58,00	—
	ResNet18 (pré-trein.)	1×10^{-4}	15	—	7,1	—	53,91	—
	VGG-Style	1×10^{-3}	62	—	30,5	—	51,81	51,19
Restrição ingênua	AlexNet3x3-FC	3×10^{-4}	37	—	29,1	—	35,79	36,19
	AlexNet3x3-GAP	3×10^{-4}	43	—	16,3	—	40,32	37,02
	AlexNetSmallKernel	3×10^{-4}	62	—	24,5	—	45,84	36,16
Tentativas de compensação	AlexNetBottleneck	1×10^{-3}	76	—	29,6	—	44,62	44,54
	AlexNetFire	1×10^{-3}	41	—	21,7	—	43,98	44,30
	Bottleneck + Fire (AlexNetFinalBottleneckFire)	1×10^{-3}	40	—	13,9	—	42,29	44,00
	Bottleneck + Residual (AlexNetFinalBottleneckResidual)	1×10^{-3}	38	—	15,6	—	45,10	45,98
	Depthwise + Fire (AlexNetFinalDepthwiseFire)	1×10^{-3}	76	—	27,3	—	43,46	42,79
Arquitetura híbrida final	Fire + Residual (AlexNetFinalFireResidual)	1×10^{-3}	77	—	28,8	—	49,79	49,20
	AlexNetFireBypass	1×10^{-3}	152	—	86,2	—	49,03	49,86
Atenção local (Eixo 7)	swin_pico_w2	5×10^{-4}	146	99	156,1	107,3	30,64	33,25
	swin_pico_w4	5×10^{-4}	143	91	150,2	97,3	31,27	33,89
	swin_pico_w8	5×10^{-4}	144	80	80,7	44,8	33,03	36,99
	swin_pico_poolmixer	5×10^{-4}	143	99	80,6	55,5	28,21	30,76
	hybrid_bottleneck_swin	5×10^{-4}	144	96	85,9	56,7	33,64	40,18
	vit_tiny	5×10^{-4}	125	89	59,6	65,8	40,35	42,44
	deit_tiny	5×10^{-4}	212	62	108,4	46,1	46,38	47,66

Tabela XII
EIXO 6 — MELHOR ÉPOCA, TEMPO DE TREINO FP32 E MÉTRICA NOS TRÊS ESTÁGIOS (MAP EM % PARA DETECÇÃO, MIOU PARA SEGMENTAÇÃO) POR BACKBONE E VARIANTE

Backbone	Tarefa	Variante	Melhor época	Tempo (h)	FP32	QAT	INT8
AlexNetBottleneck	Detecção	do zero	549	29,7	20,98%	21,00%	20,85%
AlexNetFire	Detecção	do zero	772	68,6	8,94%	6,38%	6,40%
AlexNetTV	Detecção	do zero	927	115,0	16,94%	14,73%	14,50%
AlexNetBottleneck	Detecção	pré-treinado	709	56,1	20,83%	20,15%	19,79%
AlexNetFire	Detecção	pré-treinado	596	88,4	8,93%	7,79%	7,54%
AlexNetTV	Detecção	pré-treinado	791	125,1	18,10%	17,24%	16,96%
AlexNetBottleneck	Segmentação	do zero	84	0,5	0,1784	0,1790	0,1791
AlexNetFire	Segmentação	do zero	154	1,1	0,1703	0,1712	0,1705
AlexNetTV	Segmentação	do zero	99	0,6	0,1634	0,1633	0,1621